Quarb Implementation Guide

The Quarb Project

Version: Draft 0.1 (walking skeleton)

Contents

1 About This Guide **2**

2 Architecture **3**
- 2.1 Workspace Layout 3
- 2.2 The Layers of the Engine Crate 29
- 2.3 The Execution Ladder 32
- 2.4 Current Query Subset 33

3 The Adapter Surface **67**
- 3.1 The Live Trait 67
- 3.2 The Source Boundary 76
- 3.3 Worked Example: the Filesystem Adapter 81
- 3.4 The Planned Surface 82

4 Building and Running **83**

5 Planned Integration Paths **84**

Chapter 1

About This Guide

This guide describes the **Quarb engine**: the Rust implementation of the Quarb query language, its `qua` command-line tool, and the adapter architecture that lets one engine query many data sources. The language itself — its data model, execution model, and syntax — is defined in the *Quarb Language Specification* (the `lang` sibling project); this guide covers only the implementation.

The guide tracks the code. The engine is being built as a series of thin, working slices rather than from a finished design. This document tracks that process: constructs the engine *already implements* are marked **[live]** and documented against the real code; constructs that are specified but *not yet built* are marked **[planned]** and sketch the intended surface. As each slice lands, its **[planned]** entries become **[live]**. The current milestone is a *walking skeleton*: rooted tree navigation over a filesystem.

Central concern: the adapter surface. The one design question this guide most needs to get right is the *adapter surface* — the trait a data source implements to be queried. That surface is deliberately being grown against a concrete first adapter (the filesystem) so that its shape is driven by real navigation problems rather than speculation. Chapter 3 is its home.

Chapter 2

Architecture

The engine follows a conventional compiler-style pipeline. A query string is lexed and parsed into a small AST, which the executor evaluates by driving an *adapter* over the target data:

```
[query string]
      |  lexer          quarb::lexer
      v
[tokens]
      |  parser         quarb::parser
      v
[query AST]             quarb::ast
      |  executor       quarb::exec
      v
[node set]  <-- drives --  [AstAdapter]  --> [data source]
```

The executor never touches the data source directly. It knows only the `AstAdapter` trait (Chapter 3), so the same query runs unchanged over any adapter.

2.1 Workspace Layout

The engine is a Cargo workspace mirroring the intended separation of concerns — the core engine, one crate per adapter, and the CLI:

Crate	Responsibility
`quarb`	The core engine, in layers (Section 2.2): `value`, `syntax`, `stdlib`, `pipeline`, `graph` (the `adapter` module defining `AstAdapter` is its). No I/O.
`quarb-fs`	The filesystem adapter. Depends on `quarb`.

`quarb-imap`	Remote IMAP mailboxes, queried in place: the maildir mapping over the wire (folders, messages by UID, headers as properties, `:::epoch`), transported by `curl` over subprocess (it speaks IMAP natively) — zero new dependencies. Lazy per message: one LIST, one UID SEARCH per folder, two fetches per touched message. The thread fabric is deliberately absent (indexing it means fetching every header — that is a sync; use mbsync + `quarb-maildir`). Target: `imap[s]://HOST[:PORT][/FOLDER]`; credentials via `QUARB_IMAP_USER`/`PASS` or `~/.netrc`. Read-only. Tested against a bundled deterministic mini IMAP server.
`quarb-json`	The JSON adapter (keys and indices as names, value types as traits). Depends on `quarb` and `serde_json`.
`quarb-gsheet`	Google Sheets on the `quarb-xlsx` mapping (sheets as tables, first-row headers, typed values via `UNFORMATTED_VALUE`), fetched from the Sheets API. Auth: the API rejects gcloud's `cloud-platform` tokens, so the friction-free path is an API key (`?key=` / `QUARB_GSHEET_KEY`; enough for link-readable sheets), with a properly-scoped bearer for private ones. Target: `gsheet://SPREADSHEET_ID`.
`quarb-html`	The HTML adapter (tags as names, block/inline as traits, attributes as properties, anchor fragments resolve). Depends on `quarb` and `scraper`.
`quarb-pdf`	PDF internals as the object graph: indirect objects are nodes named by their declared `/Type`, scalar dictionary entries are properties, references are edges labeled by their key (`->Kids`, `<-Parent`; array references hoist to the array's owner), with the reverse index built at load. Depends on `quarb` and `lopdf`.

quarb-text	The shared text-level model — the relational pattern for documents: the per-format crates lower their format into one block-event vocabulary (the atrep shape: kind, taxis, lemma, body, hypograph), and this crate alone implements the adapter. Sections with explicit title (::lemma) + body are derived from the flat heading stream by the outline rule; paragraphs, blockquotes (attribution as ::hypograph), lists, and verbatim blocks (:::lang) complete the vocabulary; tables denormalize into nested lists carrying the <table> trait. Bare :: is the flattened prose of the subtree. The apparatus is first class (litogramma's model): footnote/endnote callouts and bodies share their family name, resolve by (family, onym) into ->footnote/->endnote edges, and carry <deixis> (the callout), <note> (the body), and <dangling> traits; the aside family (anchored margin content) rides the same machinery without <note>, and margin-spelled footnotes carry :::form; index-mark nodes expose declared index marks (::term) invisible to the prose. Plain text (.txt, blank-line paragraphs) mounts here by default; text: forces the reading for html/md. Depends on quarb.
quarb-text-docx	The Word text-level adapter, from declared structure only: outline levels resolved through the style chain (never style names), the accepted view of tracked changes, w:tblHeader header rows, the closed cited quote-style set, both note parts (word/footnotes.xml, word/endnotes.xml), and XE field instructions as index marks. Depends on quarb-text, zip, and quick-xml.

`quarb-text-odt`	The OpenDocument text-level adapter (`.odt`, the flat `.fodt`, OpenOffice.org's `.sxw`), the Word reader's sibling: `text:h` outline levels and a paragraph style's `default-outline-level` along the parent chain, the declared list containers with their style's level kind, `table-header-rows`, the same cited quote style, inline `text:note` bodies lifted to the document end with the callout at its offset, bookmarks and reference marks as bearers or anchors, `text:a` and the reference fields as refs, index marks, emphasis and code spans from character styles' declared properties, the accepted view of tracked changes, `meta.xml` as the declared identity. Depends on `quarb-text`, `zip`, and `quick-xml`.
`quarb-text-rtf`	The Rich Text Format text-level adapter, from declared control words: `\outlinelevel` direct or through the stylesheet's `\s` / `\sbasedon` chain, the list table's level kinds behind `\ls` / `\ilvl` (and the old `\pn` group), `\trhdr` header rows, the same cited quote-style names, inline `\footnote` groups as callouts with their bodies at the end (`\ftnalt` the endnote), HYPERLINK fields as refs, bookmarks, `\xe` index marks, `\deleted` dropped, emphasis and code spans from the run's declared formatting, the code page of `\ansicpg` for the `\'xx` bytes (reported as the source's declaration), the info group as the declared identity. Its own tokenizer; depends on `quarb-text` and `encoding_rs`.

`quarb-text-doc`	The legacy Word (97–2003, OLE2) text-level adapter, the text-only first cut: the `WordDocument` stream's FIB names the table stream and the CLX, the piece table gives the text (8-bit pieces in the FIB language's code page, UTF-16 pieces as they are), paragraph and cell marks split paragraphs, a field's instruction is dropped and its result kept, the footnote text's bodies are split by `PlcffndTxt` with the footnote marks as their callouts, and `SummaryInformation` declares the identity. No outline levels, lists or tables yet (they need the property pages and the stylesheet). A plain-text file named `.doc` is told apart by the OLE2 magic. Depends on `quarb-text`, `cfb` and `encoding_rs`.
`quarb-text-epub`	The EPUB text-level adapter: container to rootfile to spine, chapters through the HTML adapter in spine order (`linear="no"` excluded); the EPUB note vocabulary arrives via `quarb-text-html`. Each chapter's ids are its own namespace (`ch2.xhtml#n1`): a reused `id` names one bearer per chapter, and a reference lands in its own chapter or the one it names. Depends on `quarb-text`, `quarb-text-html`, and `zip`.

`quarb-text-html`	The HTML text-level adapter: soup dropped (nav, chrome, scripts, forms, media), the head read for the page's declared identity (`title`, `lang`, canonical, every `meta`, `rel=tag` links, a JSON-LD block — verbatim as `:::` metadata on the document, and curated into tags-as-traits, `::category`, `::description`, `::author`, `::published`; every node answers `::href` / `::anchor` / `::link`, and `| link` rides them), wrappers transparent, headings flat into the shared derivation, trailing `cite`/`footer`/`figcaption` as quote attribution; notes only from the declared vocabularies (`epub:type` noteref/footnote/endnote/rearnote/marginalia, DPUB-ARIA `role` doc-*), endnote words landing in the endnote family, marginalia in the aside family. Depends on `quarb-text` and `scraper`.

`quarb-text-koine` The koine adapter — the hub's first rung: a koine/litogramma document (parsed by the `atrep` crate, settled by `kanonizo::tasso`) lowers into the shared block vocabulary through its dialektos-resolved sim names — enclosing headings by nesting depth, both note families and asides by name, index endos as marks, the table model (`table` / `row` / `header-row` / `cell`, the `hspan` / `vspan` monosims padding the cells a span covers, so every row is a record of the same width) and the older stichoi tables, refs keeping their key as written. The dictionary vocabulary (spec: The Dictionary Vocabulary): a lexigramma document lowers its `entry` and `sense` sims to containers of those kinds (the headword the lemma and the onym, the taxis the homograph or sense number), the inline apparatus to spans named by their sims with `::kind` / `::lang` from the genos, an `inflection` to a `<hidden>` node outside the prose, the languages line to `:::languages`, and the definition list typed `.abbreviations` (or, left untyped, the one before the first entry) to the abbreviations list (`::kind`) the labels resolve against (`-->`). Foreign formats arrive through atrep's endomorphosis importers (`atrep::endo`) into their mirror dialects — Markdown, HTML, reStructuredText, Org, djot — read by the same name-keyed mapping (flat `heading-N` endos, flat item runs wrapped, rst footnotes into the shared apparatus); Word through `atrep::docx` (`koine:report.docx`, declared structure only, beside the native `text:` reading); a Lingvo DSL dictionary (`.dsl`, `.dsl.dz`) through `atrep::dsl` into lexigramma, the `#INCLUDE`d abbreviations read beside it, under `koine:`, `text:` and `corpus:` alike (`lit:` refuses: a dictionary is no literary edition). Serves `koine:` (and `text:` on `.atd`/`.atk`, where the routes converge); `.xml` dispatches by declared identity (root namespace, DOCTYPE public id, unambiguous root element) into the TEI/DocBook/JATS/USX/OSIS importers, `?format=` forcing any format; a bare `<article>` and everything nothing imports yet — LaTeX included — refuse naming the working spelling. Depends on `quarb-text` and `atrep`.

quarb-text-latex	The LaTeX text-level adapter: the declared structural subset of a .tex source — declared sectioning commands, footnotes (the at-callout convention normalized to document-end bodies), the endnotes package on its own counter, Tufte sidenotes into the footnote family (:::form), \marginpar / \marginnote as asides, \index marks, quote/quotation, itemize/enumerate, verbatim; prose by drop list and unwrap-by-default, definition argument trains consumed whole. Depends on quarb-text.
quarb-text-markdown	The Markdown text-level adapter, mapping pulldown-cmark events directly (no HTML round-trip): heading levels, list ordinals, fence languages, and pipe tables come from the source. Depends on quarb-text and pulldown-cmark.
quarb-text-pdf	The PDF text-level adapter — the reader's model from the two declared rungs: the /Outlines tree (titles as ::lemma, destinations bounding each section's extent) and exact-baseline line geometry (bit-for-bit equal y, :::page/:::y metadata; ToUnicode CMaps decoded, one stated kern constant for word gaps). No paragraph is ever minted. Depends on quarb-text, quarb-pdf, and lopdf.
quarb-text-djvu	The DjVu text-level adapter (text:scan.djvu), by the PDF reading's doctrine: the NAVM outline is the heading ladder (sections holding their pages, front matter at the root; page/* without one); a page's zones are its tree as the text layer names them (column, region, paragraph, line; words a line's text; declared paragraphs kept, none minted); :::page/x/y/ width/height on every zone; :::text-layer on page and document; the ANT metadata as the document node's properties. Bundled, single-page and indirect documents; the text layer only (no OCR). The plain mount of a .djvu names the reading. Depends on quarb and djvu-rs (pure Rust, MIT).

`quarb-text-ofd`	The OFD text-level adapter (`text:invoice.ofd`; GB/T 33190, China's fixed-layout format), by the PDF reading's doctrine: the `Outlines` tree is the heading ladder (a section's extent from its destination — a page and a top, or a named bookmark — to the next entry's; front matter at the root; every listed page as `page/*` without one); a `line` is the text codes a page shows at one declared baseline in `x` order (a gap between two codes' extents a space; no `paragraph` minted); `:::page`, the bounding box `:::x/y/width/height` in the page's millimetres, `:::baseline`; a page's template pages join its lines; `DocInfo` (title, author, subject, abstract, keywords, dates, creator, custom data) as the document node's properties. Attachments and signatures read under the plain mount, which opens the package as a zip. Depends on `quarb`, `quick-xml` and `zip`.
`quarb-text-hwpx`	The Hangul Word Processor text-level adapter (`text:report.hwpx`; KS X 6101, OWPML — the zip of XML Hancom's office writes), in the Word and OpenDocument readings' vocabulary from declared structure alone: a heading is a paragraph whose paragraph property declares an outline level (style names never matched), a list the run of paragraphs declaring a numbering or bullet, nested by level; tables through the shared table rule (`header="1"` cells the header, column spans padded); foot- and endnotes as callout and document-end body; hyperlink fields as `ref`s, bookmark fields as anchors; tracked changes as the accepted view; headers, footers, pictures, drawings and equations not carried; the package file's metadata as the identity. The binary HWP 5 is not read yet; the plain mount of an `.hwpx` is the zip of its XML. Depends on `quarb`, `quarb-text`, `quick-xml` and `zip`.

`quarb-text-uof`	The Uniform Office Format text-level adapter (`text:report.uot`; GB/T 20916, UOF 1.0 one XML, UOF 2.0 a zip package, both by the root `uof:UOF`), in the Word and OpenDocument readings' vocabulary from declared structure alone, the Chinese element names matched as written: a heading is a paragraph declaring an outline level (on itself or through its paragraph style), a list the run of paragraphs naming an automatic numbering (a bullet at its first level unordered), nested by the declared level; tables with their declared header rows; foot- and endnotes as callout and document-end body; hyperlink regions resolved through the link set, bookmark regions through the bookmark set; tracked changes as the accepted view; anchored objects, comments and formatting not carried; the metadata as the identity. The spreadsheet and presentation modules are not read. Depends on `quarb`, `quarb-text`, `quick-xml` and `zip`.
`quarb-xml`	The XML adapter (qualified tags as names, attributes as properties, `id`/`xml:id` references resolve; strict well-formedness, no traits). Depends on `quarb` and `quick-xml`.

`quarb-relational`	The shared relational model: tables as root children, rows named by primary key, columns as properties, declared foreign keys driving `-->` resolution (chained hops instead of JOINs), `->`/`<-` crosslinks labeled by column, `<--(::fk)` reverse resolution from the declared keys (`reverse_resolve` on the relational model), and the table-naming reference label for undeclared references (required where a column declares no key). Loading is catalog- eager, rows-lazy, hops by key: a table is read whole only when a query lists it (`/t/*`, `:::n-rows`); a hop reads the rows it lands on through the driver's keyed source (`WHERE key IN (…)`, one statement per frontier; such a row carries no row id until the table is read, never a sentinel value a real row could bear), rows keep their identity when the table is later read whole, and untouched tables are never read (`:::loaded`, `:::n-loaded` report the state). Engine adapters are thin catalog drivers over it. Depends on `quarb`. The pushdown scanning round (2026-11-06): a child hop keeping a prefix reads `LIMIT k` in the whole read's order through the driver's prefix fetcher (declined where no order backs the cut), and `@| count` on a table not read is the driver's `count(*)`; a model or a multi-source mount forwards both to the source under its own node.
`quarb-sqlite`	The SQLite catalog driver: PRAGMA introspection, storage-class type mapping. Depends on `quarb-relational` and `rusqlite`.

`quarb-objstore`	GCS and S3 buckets as lazy directory trees (`gs://` / `s3://`, delimiter listings, paginated; object content fetched on first read). Composed by default in `qua`: a bucket of JSON / CSV / source files grafts, so `/data/store.json/books/*` crosses from cloud storage into the document mid-path — grafting is this adapter's point. GCS: anonymous for public buckets, gcloud-token auth otherwise. S3: anonymous only in v1 (SigV4 signing is a recorded extension — private buckets refuse).
`quarb-postgres`	The PostgreSQL catalog driver: `information_schema` introspection over `tokio-postgres` (internal current-thread runtime, synchronous surface); native bool/int/float/text, everything else `::text`-cast into the numeric-reading regime. Depends on `quarb-relational`.
`quarb-mysql`	The MySQL/MariaDB catalog driver: `information_schema` introspection over `sqlx` (internal current-thread runtime, synchronous surface; `key_column_usage` carries the foreign keys directly); integer/float/character families native, everything else `CAST(... AS CHAR)` into the numeric-reading regime. Depends on `quarb-relational`.
`quarb-mssql`	The SQL Server catalog driver: `INFORMATION_SCHEMA` plus `sys.foreign_key_columns` (parallel column ids keep composite keys paired), over `tiberius` (pure-Rust TDS, internal current-thread runtime); `bit`/integer/float/character families native, everything else `CAST(... AS NVARCHAR(MAX))`. Target: `mssql://USER:PASS@HOST[:PORT]/DATABASE`. Depends on `quarb-relational`.

`quarb-oracle`	The Oracle catalog driver: the `USER_*` dictionary views over the `oracle` crate (ODPI-C, synchronous); `NUMBER(p,0)` integer, `BINARY_FLOAT`/`DOUBLE` float, character family text, `DATE`/`TIMESTAMP` through an ISO-8601 `TO_CHAR` into the temporal regime, everything else `TO_CHAR`. Target: `oracle://USER:PASS@HOST[:PORT]/SERVICE`. Depends on `quarb-relational`.
`quarb-ldap`	The LDAP directory driver: the DIT *is* the arbor — the base DN is the root, an entry's children are the entries one level below (RDN names), its attributes are properties (multi-valued as lists), `objectClass` values are traits. DN-valued attributes (`member`, `manager`, ...) are native references — `~>` follows, `->` enumerates, `<-member` finds the groups by a filtered search. Over the `ldap3` sync client, anonymous or simple bind. Target: `ldap[s]://[USER:PASS@]HOST[:PORT]/BASE_DN`.
`quarb-cbor`	The CBOR (RFC 8949) adapter, the JSON mapping extended: non-text map keys name nodes by canonical rendering, byte strings are hex scalars (`:::length` is the raw byte count), the standard time tags (0, 1) decode to instants, other tags are transparent with the number at `:::tag`. Traits carry the CBOR type. Binary, by the `.cbor` extension. Depends on `ciborium`.
`quarb-csv`	The CSV/TSV adapter (one `row` node per record, columns as properties, empty cells as missing values that numeric aggregates skip; `:::columns` / `:::n-rows` metadata). Depends on `quarb` and `csv`.
`quarb-xpath`	The XPath 1.0 bridge, both directions: `translate` imports an XPath expression to Quarb, and `export` walks a Quarb query's reflection arbor (the locked vocabulary as the tooling surface) back to XPath — explicit refusals and divergence notes both ways. Binaries `xpath2quarb` / `quarb2xpath`; the export side is differentially tested against `xmllint`.

`quarb-jq`	The jq bridge, both directions: `translate` imports a jq filter (value-projecting; a node mode pairs with `jq 'path(...)'`), and `export` walks the reflection arbor back to a jq filter. Binaries `jq2quarb` / `quarb2jq`; the export side is differentially tested against the `jq` binary.
`quarb-bigquery`	The Google BigQuery driver: the REST API directly (`jobs.query` over a synchronous HTTP client — no cloud SDK, no async runtime), bearer token from `QUARB_BQ_TOKEN` or `gcloud auth print-access-token`, catalog from `INFORMATION_SCHEMA` including the *unenforced* key constraints (declare them and the reference machinery works). The execution ladder is a billing instrument here: BigQuery meters bytes scanned, so lazy loading, pushdown, and `--save` directly reduce cost. Target syntax: `bigquery://PROJECT/DATASET[?account=EMAIL]`.
`quarb-duckdb`	DuckDB, the fifth catalog driver — and the columnar side door: a `CREATE VIEW ... AS SELECT * FROM read_parquet(...)` makes Parquet first-class. Keys from `duckdb_constraints()` (the `information_schema` FK view reports the referencing side only), rows lazy, `raw_query` for the ladder, read-only connection. Bundled build; `.duckdb` / `.ddb` by extension.
`quarb-xlsx`	Spreadsheets (`.xlsx` / `.xls` / `.ods`, via calamine) on the relational model: sheets as tables, first-row headers as columns (positional `cN` for blank/duplicate), rows named by sheet row number, numeric cells typed. No keys — sheets declare none — so resolution needs hints. Dispatched by extension *before* the archive check (workbooks are zips; the sheets are the point).

`quarb-firebase`	The Firebase Realtime Database driver: the RTDB *is* a JSON tree, so the mapping is `quarb-json`'s, fetched lazily one node per shallow GET and cached. Properties are direct fetches (`::score` is one tiny request); `~>` resolves by hint naming a root-relative container (`::parent~>item`). Containers that refuse enumeration (permission-scoped or unbounded — the public Hacker News database's `/v0/item` holds tens of millions of keys) are *opaque*: reachable by name, empty to wildcards — served by the engine's new `children_named` fast path. References are *declared client-side*: a refs document (`{"refs": {"parent": "item", "kids/*": "item"}}`, supplied via `qua --refs`) is the portable mini-catalog a schemaless substrate cannot hold — with it, bare `~>` resolves and `->` crosslinks enumerate (array fields one probed edge per element); an inline hint overrides. Reverse resolution stays empty (it would scan what opaque containers refuse; server-side `.indexOn` queries are the recorded v2 path). Target: `firebase://HOST/PATH[?auth=...]`. Read-only.
`quarb-tree-sitter`	Source code as its literal syntax tree — the syntax level (né `quarb-code`): node kinds as edge names (`//function_item`), tree-sitter *fields* as properties (`::name` on a function is its name), source text as the value, `:::start-line` / `:::kind` / `:::field` metadata; named nodes only. Rust / Python / JavaScript / C by extension, compile-time grammar set. Grafts under composition — `--graft` queries every function in a directory tree at once.

`quarb-code`	Source code at the code level: declaration identifiers are node names (`/lexer/lex`), anonymous constructs keep a normalized cross-language vocabulary (`if`, `switch`, `for`, `call`), everything else dissolves (children hoist). Curated traits (`<function>`, `<type>`, `<loop>`); `::signature` / `::doc` / `::callee`; raw backend kind only at `:::kind` (aliased, closed surface); `->definition` file-local crosslinks, one declaration list per identifier: a qualified callee (`Type::new`, `obj.helper`) prefers the declarations under that qualifier, an unqualified one fans out. Opt-in via the `code:` target prefix; `code:DIR` grafts every supported source leaf — filesystem into declarations, one namespace.
`quarb-compose`	Adapter *composition*: wrap any adapter and a leaf whose name or content parses (json / xml / html / csv, else a sniff) grafts its parsed arbor as children — `/data/store.json/books/*` crosses from filesystem into JSON mid-path. Marked-up members graft at the DOM by default or, under `DocumentGraft::Text` (`text:` on an archive), as the text level; the source-level grafts (tree-sitter, the code model) sit behind the default `source` feature so the wasm build can leave them out. Grafts are lazy (read + parse on first entry), the inner root is identified with the outer leaf (no phantom node), and locators mark the boundary jar-style (`store.json!/books/1`). `qua`: default for archives, `--graft` for directories.

`quarb-archive`	Zip, tar and 7z archives as directory trees (entries as nodes, content as value, `:::size` / `:::compressed`), every format through a pure-Rust library so the wasm build reads them too: zip members deflate, deflate64, bzip2 (`bzip2` with `libbz2-rs-sys`), lzma (`lzma-rs`); tar plain and under gzip (`flate2`), xz (`lzma-rs`), bzip2, zstd (`ruzstd`); 7z (`sevenz-rust`, LZMA/LZMA2). The format is told by its magic bytes. Zip members in zstd or xz and rar would need a C library and are not read by default: rar rides the opt-in `rar` feature over the `unrar` crate (rarlab's library under its own licence), on for `qua`, off for the playground and the extension, and a RAR without it is refused naming the feature. CHM help files read through `libchm` (pure Rust, `ITSF`): members under their paths, the system records left out, a page without a `<meta charset>` decoding under the `#SYSTEM` locale's code page. An MHTML web archive (`.mhtml`, `.mht`, or MIME header lines) reads through an in-crate RFC 2045/2046 splitter: each part a member under its `Content-Location` path, transfer-decoded, its charset the declaration. Members decode under the mount's source-boundary rule.

quarb-web

The *web level*: a site as an arbor — /sites/<host>/pages/<path> for every page (directories from the URL paths, <dir> / <page> / <redirect>; a redirect stub stands for a page and is not one, as a symlink: //redirect reaches it, //page never does, a link to it lands on its target and it takes no rank; a relative rel=canonical is joined against the page's URL), the declared categories and tags as rows of their own containers, every hyperlink of the DOM resolved into ->link / <-link (->category, ->tag, <-page, <-category), the degrees, ::pagerank, ::redlinks (the distinct content links to pages the site lacks, the chrome's rows never counted), and <orphan> as columns; a page is no backlink of its own, its fragment links land on its sections; each page grafts at the text level on entry, and a prose ref that leaves its page resolves to the page it names. Rows answer to their kind (//page), their segment, its stem, their title. A WebStore contract behind it; stores: in-memory from a tarball (archive, the wasm Site) or a directory (fs), SQLite (sqlite), PostgreSQL (postgres; COPY export, deferred foreign keys), one schema in two dialects. A planner over the reflected query answers on three rungs — Full (a bare count, every conjunct exact), Prefilter (a superset of the enumerating step's matches; the engine re-runs the query over the level scoped to the candidates), Scan — with the text prefilter resting on page_text.plain = the graft's own plain_text(). The four adapter hooks (document_by_ref, reverse_resolve, descendants_named, prefetch_links) let a store answer where the engine would walk. quarb-web-ingest (cli): crawl a directory or archive into a store, wikipedia the Wikimedia Enterprise HTML dumps (streamed; links by URL then through redirects; --tree=path|category), analyze, index (pg_trgm). qua: web:<dir|archive|site.db|postgres://…>, --diagnostics=info / --no-pushdown.

`quarb-trace`	The `quarb/trace` viz contract as Rust: the payload types (arbor snapshot, steps, threads, snapshots, window and stubs, positions, ribbons, lookahead verdicts, results) and their kaiv writer; no dependencies, no reader. `quarb::trace::trace` fills one by *replaying* the query rather than instrumenting the evaluator: one hop per live thread through the evaluator's own `navigate_paths` (candidates with the predicates stripped, survivors with them — the fork, the verdicts, the walked edges), `apply_command` per thread for filters and over the whole context for aggregates, a hand-iterated walk for quantified groups (one frame per depth), then checks the final context against a plain evaluation. A staged suffix marks the steps beyond the committed query (a filter preview); a window selects the slice of a large result list to embed and what the stubs on either side measure. `Doc::trace` in `quarb-session` is the entry; the renderers read the kaiv document (quarbopsis: `quao trace`).
`quarb-maildir`	Maildir (`cur/`+`new/`) and mbox mailboxes: headers as properties (lowercased), the body as the value, `:::epoch` parsing the Date header for range predicates. The thread graph is reference fabric: `::in-reply-to~>` walks to the parent message, `<-in-reply-to` finds the replies, via a Message-ID index. Never touches flags; read-only.
`quarb-firestore`	Google Firestore (native mode): collections and documents alternate as the arbor's levels; map and array fields descend as children, subcollections beside them. `referenceValue` fields are *native* references — the first remote adapter where `~>` and `->` need neither a schema nor a refs file. Lazy per level, REST direct, token via `QUARB_GCP_TOKEN` / gcloud. Target: `firestore://PROJECT/DATABASE[?account=...]`.

`quarb-datastore`	Google Cloud Datastore (Firestore in Datastore mode): kinds and entities, the most relational of the document stores; `keyValue` properties are native references (`~>`/`->` schema-free). Kind list via a `__kind__` query, entities via cursor-paginated `runQuery`, lazy per kind. Target: `datastore://PROJECT[/DATABASE][?account=...]`. Both drivers send the `x-goog-request-params` routing header named databases require.
`quarb-mongodb`	MongoDB: collections hold BSON documents, named by `_id` (ObjectIds as hex); embedded documents and arrays descend as children. DBRef fields (`{$ref, $id}`) are native references (`~>`/`->` schema-free). BSON dates arrive as instants; ObjectId and Decimal128 as strings. Lazy per collection (one `_id`-sorted `find`), reference targets by single `findOne`s; official Rust driver, sync API. Read-only. Target: a standard connection string with the database as the path, `mongodb://HOST[:PORT]/DATABASE[?options]` (or `mongodb+srv://`); a missing `serverSelectionTimeoutMS` defaults to 5 s.
`quarb-kubernetes`	The Kubernetes API, which is already a tree: the root holds one listing per discovered resource (cluster-scoped kinds plus the all-namespaces view, kubectl's `-A`); each namespace object carries its scoped listings beside its own fields; objects descend as their JSON, RFC 3339 strings as instants. `ownerReferences` are native references (`::owner~>` climbs Pod → ReplicaSet → Deployment schema-free; `->node`, `::namespace~>`); unknown property names fall through to `metadata.labels`. Lazy per listing, discovery at connect. Transport is `kubectl get --raw` over subprocess (git-adapter posture, zero new dependencies; `QUARB_KUBECTL` overrides the binary), so contexts and auth behave exactly as kubectl does. Read-only. Target: `k8s:` / `k8s:CONTEXT` (alias `kubernetes:`).

`quarb-gcplogs`	Google Cloud Logging as a bounded snapshot — never a tail: the target must carry `since=` (a gcloud freshness duration or an ISO instant) and/or `limit=`, and an unbounded mount is refused. Entries oldest-first at the root as `/entry` children (the `/row` convention); severity as text property, lowercased trait (`/entry<error>`), and ordered numeric rank (`:::severity`, GCP's enum values); timestamps as typed instants; `::trace` the bare trace id — the `<=>` join key across services. Unknown property names fall through payload top level → labels → resource labels → httpRequest. `filter=` passes a raw Cloud Logging filter to the server (the mount-side cut); `until=` joins it as a timestamp clause. Transport is `gcloud logging read --format=json` over subprocess (kubectl posture; `QUARB_GCLOUD` overrides — the bottled test fixture's hook; `account=` picks the gcloud account). Read-only. Target: `gcl:PROJECT?since=1h&filter=…&limit=N` (alias `gcplogs:`).
`quarb-cwlogs`	AWS CloudWatch Logs, the gcplogs design sheet over SigV4 (shared `quarb-aws` signing, credential chain, region resolution): bounded snapshots (`since=`/`limit=` mandatory), log groups as short-named root children with lazily fetched `/entry` events (or one group's entries at the root when the target names it), JSON messages decoded with property fallthrough (`::level`, `::requestId` — the cross-group join key), `filter=` as a server-side CloudWatch filter pattern, `:::group`/`:::stream`/`:::received`/`:::id` metadata. `endpoint=` overrides the URL (LocalStack; the bottled `cwlogs-server.py` fixture). Read-only. Target: `cwl:[GROUP]?since=1h&filter=…&limit=N` (alias `cloudwatch:`).

`quarb-azlogs`	Azure Monitor Logs through the Log Analytics KQL endpoint — the adapter composes the KQL itself (bounded table scan, `filter=` as a `| where` clause, `take` from `limit=`); `table=` and a bound are mandatory. One table's rows at the root as `/entry`, a comma list as root children per table. Columns are properties (`datetime` as instants, `dynamic` decoded into subtrees with fallthrough); `::OperationId` is the correlation key; the default value is `Message` or the `Name`+ `ResultCode` request line. Bearer token from `$AZURE_LOG_TOKEN` or the az CLI (`QUARB_AZ`); `endpoint=` overrides (the bottled `azlogs-server.py`). Read-only. Target: `azl:WORKSPACE?table=…&since=1h&limit=N` (alias `azlogs:`).
`quarb-cflogs`	Cloudflare edge logs, Logpull-shaped: `GET zones/{zone}/logs/received` NDJSON, one flat record per request, fields as properties under their own names plus minted `::ray` and `::timestamp`; the default value is the request line; `:::cache` the cache status. `since=` is mandatory (Logpull demands a range; `until` clamps a minute behind now). The Ray ID is what the origin sees as `cf-ray`, so mounting the edge next to origin logs makes request reconstruction cross-provider — one `<=>` on the ray, another on the origin's own trace. `$CLOUDFLARE_API_TOKEN`; `endpoint=` replays Logpush archives (the bottled `cflogs-server.py`). Read-only. Target: `cfl:ZONE_ID?since=1h&limit=N` (alias `cflogs:`).

`quarb-ddlogs`	Datadog Logs through the v2 search API (`POST /api/v2/logs/events/search`, cursor-paginated): bounded snapshots (`since=`/`limit=` mandatory), `query=` as Datadog's own search syntax run server-side. Severity is Datadog's `::status` plus the family trait and ordered `:::severity` rank; `::trace` is minted from `dd.trace_id`; tags split into fallthrough properties (`env:prod` answers `::env`); custom attributes navigate as children. Auth `$DD_API_KEY`/`$DD_APP_KEY`, site from `site=`/`$DD_SITE`; `endpoint=` overrides (the bottled `ddlogs-server.py`). Read-only. Target: `ddl:?since=1h&query=…&limit=N` (alias `datadog:`).
`quarb-github`	GitHub, three shapes in their native registers. The tree mirrors github.com URLs: users/orgs at the root (addressed by login — the root does not enumerate), repositories as their children, `files` / `issues` / `pulls` / `releases` sections below; file content and issue bodies are node values. The social graph is edge fabric, API-backed both ways: `follows`, `starred`, `owner`, `parent` (`<-parent` = the forks), `author`/`assignee`, `org`, and `contributor` — the first adapter with true `backlinks`, and its contributor edge carries data (`$-::contributions`). Counts and scalars are properties (`::stars`, timestamps as instants); flags, issue labels, and repo topics are traits (`<fork>`, `<label:bug>`, `<topic:cli>`); an organization answers the `<org>` shorthand and the forge-neutral `<group>` alias. Direct addressing skips enumeration (`/rust-lang/rust/issues/12345`, closed included; listings show the API's default open set). Transport is `gh api` over subprocess (`QUARB_GH` overrides); lazy, cached, read-only. Target: `github:[OWNER[/REPO]]`.

`quarb-gitlab` GitLab, whose defining shape is the *deep tree*: groups nest arbitrarily and mix subgroups with projects as children, which the REST API walks by URL-encoded full paths and the arbor restores to a path — `/tesslab/instruments/gauge/issues/7`, one GET per segment. Project sections: `files` (content as value), `issues` / `mrs` (the open sets; direct addressing by iid reaches any state), `releases`, `pipelines` (jobs as children; statuses as traits — CI is half of what a GitLab project is). Code over social: no follows, stars as a count; the edge fabric is `member` (with access-level edge data — `$-::access`, `$-::role`), `parent` on forks (`<-parent` = the forks), and `author` / `assignee` / `reviewer`. Labels (scoped included), topics, visibility, and statuses are traits; a project answers the forge-neutral `<repo>` alias (`//*<repo>` sweeps either forge identically). Transport is `glab api` over subprocess (`QUARB_GLAB` overrides; hosts via glab config), so self-managed instances work as glab does. Read-only. Target: `gitlab:[PATH]`.

`quarb-git` The git repository driver: the object DAG as the arbor. `/branches`, `/tags`, `/HEAD` (refs alias their commits — same properties, same children), `/commits` (rev-syntax aliasing: `/commits/'HEAD~2'` navigates without enumeration; `/commits/*` batches one `rev-list --all`). A commit's children are its *tree* — time-travel file access, blob content as the node value. References are inherent: `::parent~>`, `->parent` (merges fan out), `<-parent` (children, from the enumeration). The diff surface: tree entries carry the `<changed>` trait when their path is in the commit's diff vs its first parent (trees by prefix), so `git log -- path` is `/commits/*[/path<changed>]`; commits answer `::changed` (the touched paths, deletions included) and `:::n-changed`. Plumbing over subprocess — no libgit2, zero new dependencies. Target: `git:PATH`. Read-only.

`quarb-atrep`	The atrep semantic-document adapter — the OQL homecoming. Mounts a deltos (`.atd`) or kanon (`.atk`) *through its dialektos*: sims named by their dialect names (`//section`, `//footnote`), genoses as traits, `::onym`/`::taxis`/ `::lemma` projections, deixes and onym-monosims as typed crosslinks with a precomputed reverse index (the first adapter to prebuild both `links` and `backlinks`), `<dangling>` as a broken-cross-reference linter. Mounting runs `kanonizo::tasso` (ordering only — onyms stay as authored; transclusion stays visible as edges). Depends on `quarb` and `atrep`.
`quarb-serve`	The *serve protocol* — the extension surface for tools the engine never links: a tool implements `AstAdapter` over its own model and hands it to `serve()` (three lines; non-Rust tools implement the protocol directly — nine operations over line-oriented JSON), and `qua serve:COMMAND` spawns it and mounts it beside every other source. LSP's shape: the protocol is the plugin system. Two wires, one protocol: the handshake is JSON (the universal floor) and advertises formats; every first-party server offers *kaiv* (`.daiv` frames, natively typed leaves under `/@serve`) and the client upgrades to it — the house format is the default wire, JSON the foreign-tool floor. Read-only by construction (no mutating operation exists). First external user: `cuj quarb-serve` (the cuj vault, mounted without qua knowing cuj). Reference server: `quarb-serve-json`; identity-tested against the direct adapter.

`quarb-sql`	The SQL bridge, both directions: `translate` imports a `SELECT` statement (WHERE, one JOIN via correlation and the witness, GROUP BY/HAVING, ORDER/LIMIT/DISTINCT, `LIKE` as case-insensitive regexes), and `export` walks a Quarb query's reflection arbor back to `SELECT` — witness joins become `JOIN ... ON`, grouped reductions become `GROUP BY`/`HAVING`; `~>` chains are refused by design (foreign keys live in the schema, not the query — spell `<=>` to export). Both directions differentially tested against SQLite; binaries `sql2quarb` / `quarb2sql`.
`qm`	The mailbox interface for the terminal — the first *library-path* client beside `qua`: it opens the Maildir adapter once per invocation, addresses messages as node handles, and phrases its subcommands (`ls`, `read`, `thread`, `from`, `grep`, `count`) as queries where a query is the natural shape, with `qm query` as the raw escape hatch. Actions are the tool's, not the engine's: `qm seen` performs the Maildir flag rename itself on the file handle the adapter exposes (`path_of`).

qua	The command-line tool. Picks the adapter from its argument (a directory is filesystem; `.csv`/`.tsv` is a table; a file or piped stdin is XML by extension or a leading `<?xml`, HTML by extension or a leading `<`, else JSON) and prints results. `--xpath` / `--jq` / `--sql` translate the query from XPath, jq, or SQL first; database inputs get *pushdown* — a query whose constructs are all in the verified-safe set (filtered selections, whole-table aggregates, witness joins) compiles to SQL and runs on the server, with rows ordered by the catalog key so output is identical to the scan path (`--no-pushdown` opts out; any refusal or error falls back silently, and `--diagnostics=info` prints the decision — the pushed SQL, or the construct that kept the query on the scan path). When full pushdown refuses, *partial* pushdown pushes the leading row predicates as the table's fetch filter and runs the original query over the reduced set (gated: single reach, no crosslink/resolution axes, no metadata — anything that could observe the filtering). `--save FILE` materializes a result — `.db`/`.sqlite` as a table (`--as` names it), `.json` as a JSON array — both first-class inputs for later queries (the reference syntax is the existing file/mount mechanism); `--daiv` emits canonical kaiv (typed leaves with per-value provenance; needs the sibling `kaiv-rs` checkout).

Adapters are separate crates by design: `quarb-fs`, `quarb-json`, `quarb-html`, and `quarb-xml` share nothing but the `AstAdapter` trait, and a consumer pulls in only the adapters it needs. Each new adapter has confirmed the surface without changing it — the filesystem, a JSON tree, an HTML DOM, and an XML document are genuinely the same arbor to the engine.

2.2 The Layers of the Engine Crate

The `quarb` crate is one crate in layers, each a directory under `quarb/src/`, each allowed to reference only the layers below it:

Layer	Holds
`value`	The value space: null, bool, int, float, text, list, record, table, foreign; numbers, quantities, locales, transliteration, encodings. What an AST literal holds. No node.
`syntax`	The language as text and tree: lexer, parser, AST, unparser, highlighter, completion, the function registry. One grammar for the calculator and the graph language.
`stdlib`	The functions over values, reached by name from a call, with the arithmetic and comparisons the operators run.
`thread`	The forking machine: a list of threads and three moves (fork, terminate, the whole-list map); every layer above hangs its own per-thread state on the thread, copied on a fork and never shared afterwards.
`pipeline`	Commands, control operators, bodies, environments, the registry and memo of a run, provenance.
`graph`	Arbors, anchors, steps, correlation, adapters, identity, tabula reading, reflection, the trace. Everything that knows a node.

`lib.rs` is the facade above all of them: the public paths (`quarb::adapter`, `quarb::Value`, `quarb::run`, ...) are re-exports and did not move. The dependency direction is checked, not assumed: `scripts/layers.py` resolves every `crate::` path in a layer's files to its layer and fails on a reference upward that `scripts/layers-debt.txt` does not record. The record is the debt still being paid and is meant to burn down to nothing; a change that adds an upward reference fails the check, and one that removes some must update the record in the same commit. Six references remain: the parser's default reflector (a test convenience), the library's two kaiv arms, and the graph tests' reach into the facade.

The thread and its attachment. The thread (`thread/mod.rs`) is generic over what the layer above hangs on it: `Thread<X>` carries a register, a topic with its provenance, the members of a group, the captures of the last match, and one `ext: X`. The `Attachment` trait names what a scope may see of the attachment beside the attachment itself: a `View` (the graph's: whether the thread stands on a node), a `Crossing` (the edge a hop is crossing, for `$-`), a `Witness` (what a correlation search tries in place of the thread's own bindings), and a `Key` (what a context is indexed by). The calculator's attachment is the unit type, so a calculator thread is the register and the topic and nothing else. The graph's is `GraphExt` (`graph/navigate.rs`): the node, the correlation bindings and the arrived edges, and the capsa of the spec is `Thread<GraphExt>`. The walk's frontier carries one attachment per path, so a predicate pass borrows it and clones nothing. A `Position` is a number the thread and the pipeline carry and compare; only the graph reads it

as a node.

The register is the whole memory (spec: The Capsa): one sequence of regulae, a regula any value, node references included. A mark `.name` files a reference to the position (`Value::Node`, minted by `graph::navigate::node_ref` with the node's coordinate and its locator), so there is no mark store beside the register and one name pool over both kinds. The reads split by side: `$. @. %. %.*` see the regulae that are not references, `$$. @@. %%. %%.*` the references, positions counting on each side; a name reads whatever was filed under it last (`pipeline/register.rs`). The dedup key of a walk is the node and the merged sequence, so the interleaving of values and references is part of a thread's state. The cursor is the scope's business, not the thread's: `$$_` (`Operand::Position`) reads the node the scope stands on, `$$__` (`Operand::Invoking`) the scope's `outer`, set when a query body opens, and `$$0` (`Operand::Driver`) the scope's `driver`, set by the join condition alone and refused by the parser anywhere else. Bare, each is a reference; a reference read as a value prints and exports as its locator, equals a reference to the same node, and resolves to it under `-->`.

The host. The pipeline evaluates commands and operands against a `Host` (`pipeline/mod.rs`), the trait that holds every question the pipeline cannot answer from a value alone: the root attachment, a property of a node, the provenance at a position, a node's scalar, its rendering as JSON, XML or a table, the navigation of a step, the edges of a node. Every adapter is a host (`graph/host.rs` implements the trait for any `AstAdapter`), and so is the calculator's `Calc`, whose attachment is the unit and whose node questions are refused by name. The generic operand evaluator (`pipeline/operand.rs`) runs on any host; the graph keeps the node-reading half (`graph/operand.rs`) and wraps the generic one with a witness slot for correlation. The registry tags every command with the layer its answer comes from (`Layer::Value` or `Layer::Node`); the calculator's entry refuses a pipeline that names a node command before it runs, as it refuses a head that is not bare.

The parser's reflector. The parser expands macros through an `Expander` (`syntax/expand.rs`), a trait with the usage questions a macro body needs answered; the graph's `Reflector` implements it over an optional adapter, so the syntax layer never names the evaluator. The ambient state a run needs below the pipeline (the collation, the sentence bonds, the refusal channel the library reports into) lives in `value/ambient.rs` as a stack entered per run; the run options the graph adds (the plan, the budget, the reference targets) stay in `graph/run.rs`.

2.3 The Execution Ladder

Database adapters evaluate on a three-rung ladder; every rung is *output-identical* to the one below it (differential tests hold this), so the rungs trade only latency and transfer:

1. **Full pushdown** (`quarb_sql::pushdown`, the exporter's strict mode): the whole query compiles to SQL when every construct is in the verified-safe set — WHERE-filtered selections, whole-table aggregates, witness joins. Strictness is the point: anything whose SQL semantics are not provably identical (LIKE case folding, truthiness, GROUP BY/DISTINCT order, collations, unordered LIMIT) refuses. Row order is part of the contract — the plan names the result table, and each driver appends `ORDER BY` its key from the catalog. Column types are part of it too: Quarb compares by value (numeric-looking text meets a number as a number, a number and other text never meet), SQL by the column's type, and SQLite and MySQL coerce a mixed comparison by rules of their own. A numeric-looking *text* literal therefore never pushes; for the rest the plan records what it assumes (`ColumnTypes`: the columns it compares with a number, those it compares with text), and qua asks the driver's catalog (`column_catalog`: SQLite's affinity, MySQL's `data_type`) before using a plan, full or partial; an unmet assumption scans, and `--diagnostics=info` names the column. The strictly typed engines reject the mismatch themselves, which falls back the same way. What remains outside: a SQLite cell that violates its column's declared type. Measured: COUNT over 500k PostgreSQL rows, 2.57 s scanned vs. 0.018 s pushed.

2. **Partial pushdown** (`quarb_sql::partial_pushdown`): the leading run of strictly-translatable row predicates becomes the table's fetch filter (each driver's lazy fetcher accepts a per-table WHERE), and the *original* query runs unchanged over the reduced set — the engine re-applies the pushed predicates, a no-op on pre-filtered rows, so there is no query rewriting. Gates: leading run only (a positional predicate before an expression one sees unfiltered rows), a single `/table/*` branch with no correlations, the table reached exactly once, no crosslink/resolution axes, no `:/:::` metadata (each would observe the filtering). Since the JSON-column round (2026-07-31), refused predicates in the leading run degrade to *superset prefilters* instead of aborting the plan: fixed-path JSON comparisons emit per-dialect forms proven never to drop a matching row (SQLite typed extract with a `json_type='text'` escape hatch, PostgreSQL `jsonb_typeof` CASE with `::float8`; existence and `!=` floor at `json_type IS NOT NULL`) — extras are re-checked by the re-run, and a server-rejected prefilter surfaces through an eager prefetch and falls back to the scan rather than answering empty. Full JSON string-equality pushdown carries the dialect's validity guard

(`json_valid` and kin) and refuses numeric-looking literals (Quarb's `value_eq` coerces those numerically; no extractor matches that). Measured: unpushable grouping over a 1%-selective filter, 2.31 s vs. 0.081 s; JSON numeric compare over 200k rows, 1.46 s scanned vs. 0.25 s prefiltered.

3. **Lazy scan**: catalog-eager, rows-lazy per table (Section on `quarb-relational`); correct for everything, always the fallback.

`qua --diagnostics=info` prints the decision chain; `qua --no-pushdown` pins the scan rung; `qua --save FILE [--as NAME]` materializes any result as a first-class input (`.db` table / `.json` array) so expensive reductions run once and iteration proceeds locally. For metered engines (a future BigQuery adapter bills by bytes scanned), the ladder's transfer reductions are cost reductions.

2.4 Current Query Subset

The engine implements essentially the whole language. A query is a root-anchored sequence of hops, each an axis followed by a name:

Form	Meaning
`/name`, `//name`	**[live]** child / descendant named `name`
`\name`, `\\name`	**[live]** parent / ancestor named `name`
`>name`, `<name`	**[live]** next / previous sibling
`>>name`, `<<name`, `>>?`, `>>!`, `<<?`, `<<!`	**[live]** sibling reach: all following / preceding siblings at any distance, with proximal / distal quantifiers (matcher before reach, mirroring `//?`); covers CSS `~` and XPath following/preceding-sibling
`//?`, `//!`, `\\?`, `\\!`	**[live]** proximal (nearest) / distal (farthest) reach
`^`, `name$`	**[live]** root anchor / leaf anchor
`*`, `*.rs`, `~(re)`	**[live]** wildcard, glob, and regex name matching
`<code>`, `<a|b>`, `<a><b>`	**[live]** trait filters (alternation, conjunction)
`::`, `:key`, `:::key`	**[live]** property, core-metadata, and adapter-metadata projection
`[:::size > 1K]`, `[/child]`, `[n]`, `[-1]`, `[2..5]`	**[live]** predicates: comparisons (`= != < <= > >=`), substring `*=`, regex `=~`/`!~`, structural conditions, `and`/`or`/`!`/`not`, positional index and range (negative counts from the end)

`::price * ::qty`, `7 div 2`, `idiv`, `mod`	**[live]** value expressions: arithmetic over numeric readings, in predicates, pipeline commands (`| ::a * ::b`), and pushes (`| .total(::a * ::b)` — a computed column); null propagation, exact integers, overflow to float
`//a || //b`, `| count`, `| sort | join(",")`	**[live]** union, and pipelines over the standard library's command set
`->target`, `<-target`, `->*`	**[live]** crosslink navigation (filesystem: symlinks)
`(/ul/li)+`, `(/p|/div)`, `(->mgr){1,3}!`, `/{2}`	**[live]** path patterns: groups, subpath alternation (strict and tolerated forms), quantifiers (`+ * {n} {m,n} {m,}`) with proximal/distal suffixes, the pattern dot wildcard, and the bare-operator sugar. A set walk (an open quantifier with a floor of at most one, under all matches or the proximal reach, no push or mark, every alternative one step, a guard that does not read the walk: `set_form`, `guard_reads_walk`) reaches each node once, at the fewest rounds, and runs to its fixpoint; every other walk is exact, one acyclic walk per history (no node revisited within one walk, the anchor included), its rounds capped by the adapter's quantifier bound (default 32; `--quantifier-bound` overrides). Both are capped by the frontier bound on the paths in flight per round (default 100 000; `--frontier-bound`). Under a budget the proximal reach keeps the survivors of least total, the tie set, as the distal keeps those of greatest (ruling 2026-11-10). The search by total, least first and each node settled once, runs where it is exact (`by_total` in `expand_group`: one brace, no floor, a rule other than `min`, one-step alternatives that record nothing, a guard that reads the node alone) and ends at the first total past the answer's; any other form runs the exact walk and keeps its lightest survivors. A walk cut at the quantifier bound below the answer's total refuses
`| upper`, `@| count`, `| .n`, `| $.n`	**[live]** per-capsa transform, whole-context aggregation, register push and recall

`@| sort_by(::age), @| top(3, ::fare), @| [..n], | [::age > 30]`	**[live]** keyed aggregates (`sort_by`, `unique_by`, `min_by`/`max_by`, `top`/`bottom`; capsa-preserving, missing keys do not compete), whole-context positional selection, and per-capsa filters
`| .k(::k) @|.k top(2, ::v), $ord`	**[live]** the fold by a regula (one partition per value of the pushed key, the command per partition; nothing is pushed on the folded thread, the folding regula stays where it was pushed and a reduction keeps it with what its inputs share, each member holding the group's value there, ruling 2026-11-10; `group` retired 2026-11-04), `ungroup` for windows, and the `$ordinal` operand (position in the current context, scope-threaded into filters)
`| %., qua q.quarb`	**[live]** the register record view (named regs as a record: the row-wise table view, subsuming the planned `@.|`) and reflection: `.quarb` files open as the query's own arbor (`quarb::reflect`; vocabulary locked v1, enforced by the conformance corpus in `tests/vocabulary.rs`, version on the root as `:::vocabulary`)
`def &f($x): ...;, &f(::age), --defs, --expand`	**[live]** fragments: parse-time expansion with call-by-name `$param` forms, defs files (`#` line comments stripped in `parse_defs`; the block comment `!! ... !!` is whitespace to the lexer wherever it stands, ruling 2026-11-06), and the unparser (`quarb::unparse`) rendering any parsed query back to canonical text (`--expand` = macroexpand); correlation chains and inline-pipe splices as bodies; `rec`/`group` field-name collisions are parse errors (`validate_record_convention`)

`| .n(cc),@| .total(count) @| %., @| %(total = count; unique = unique @| count),sort(::name; --locale=ru)`	**[live]** subcontext bodies: every parenthesis not after a command name is a pipeline fed by its control operator; a headless first command applies to the body's seed (`Operand::Seed`), a bare word names a command (unknown refused by name), a command's own parentheses hold operands then one option segment (`Arg::Opt`, shell spelling, `-=` the void option; undeclared options refused against the registry's `options`); the aggregate push files once on every capsa and passes the context on, `@| %(…)` and `@| %.` read the context as a whole, every reduction keeps the regulae its inputs share (`shared_register`); reflection kinds `cmd` (né `func`), `agg-push`, `agg-record`, `agg-recall`, `seed`, `option`
`| &$.f,\| &$.f!,| &$.f#`	**[live]** the held quarq spliced and applied (spec: Quarqs, the held quarq's appliers; rulings 2026-11-10). `&$.f` is the body written in place: where every thread of the context holds the same quarq (the bodies one object, or one canonical text) it runs once over the whole context, so `@_` and an aggregation inside it see what they would see written there; threads holding different quarqs, or a projector, are read each on its own. `&$.f!` and `&$.f#` apply per thread on the capsa as it stood before the regula holding the quarq: the body's thread carries that capsa as its register (the applying register is put back after the body, the body's own pushes appended), so every command of the body reads it and the body sees what it pushed. There is no window mark on the thread. `#` starts from the topic and the position of the push (`Reg::site`, `Reg::placed`; an unplaced push photographs no position) and returns to the applying position unless the body walked. Tests: `qua/tests/review_1110.rs`, `qua/tests/quarq.rs`

```
qua '…| sh:tr(a-z;
A-Z) @| go:sort(-r)'
go=exec: /go/bin
sh=shell:
```

[live] environment commands (`quarb::env`): an environment is a namespace of commands the host mounts beside the built-ins, `ALIAS=SCHEME:TARGET` from the command line only (never a model file, a defs file or query text; `--allow-shell` is `sh=shell:` under its older name); the kinds `exec:` (a directory of programs or one program) and `shell:` (the user's shell) with the text framing (`|`: once per capsa, the topic's text on stdin, stdout minus one newline the value, a failed call null; `@|`: once over the context, one line per capsa on stdin, scalars as text and records or lists as their JSON line, every stdout line a capsa carrying the shared regulae, a failed call a refusal); operands as words in order, options first in shell form, the void option onto nothing; an unknown name refused with the alias (`Environment::resolve`; the shell's 127); the gate (`exec::gate`) refuses an unmounted alias before the run; reflection `cmd`/`agg` carry the alias as `lib` (spelled `env` in 0.36.0); `python:` is named and refused until the native framing lands

`| morph:lemma, | morph:parse(-n=1), @| morph:lemma, cargo build -p qua --features morph`	**[live]** command libraries (`quarb::library`): the `CommandLibrary` trait (prefix, declared commands, call, run-time requirements and their availability), a `Synopsis` per command (operands in order with accepted values, options with short and long names and whether a value follows, the pipe or pipes, the input slot by position or option name, the return), checked by `exec::gate` before the run with the synopsis quoted in every refusal; the input slot filled by the pipe unless `$_` places it; options as keyword arguments; after `@|` the stream is the value and a returned list is one capsa per element with the shared regulae; a failed call null after `|` (text on stderr once) and a refusal after `@|`; a node crosses as the record of its name, text and properties. The pilot `quarb-lib-morph` (pymorphy3 through pyo3 `auto-initialize`, feature `morph` on qua, a Python-free default) with `quarb-pyvalue`, the value mapping shared with `quarb-py`; linked libraries are listed in `RunOptions::libraries`, and a library's prefix refuses an environment mount of the same alias

`@| scipy:mannwhitneyu, @| scipy:linkage(--pairs), @| sklearn:pca(-n=2), --features scipy,sklearn`	**[live]** the stream as a table (`quarb-lib-scipy`, `quarb-lib-sklearn`): the glue reads a list of numbers as a sample, a list of records as a matrix (numeric readings per value as the columns in record order, the first text field the row label carried back on every returned row), a list of lists as rows; a k-sample test splits one label and one value per record into samples in the order the labels first appear; `--pairs` reads two text fields and one numeric by position as row, column, entry; a hole (a column missing in one record, a value with no numeric reading, a null) refuses by name unless `--nan-policy=omit`; every test returns `%(statistic; pvalue; …)` and the other fields keep scipy's and scikit-learn's names (`df`, `dof`, `rvalue`, `n_components`); `--digits` rounds to significant digits; the shared call plumbing (`quarb_pyvalue::PyGlue`: the module compiled once, the slot filled, options as keyword arguments with `-` spelled `_`, a Python exception a failed call) also carries `quarb-lib-morph`

```
@| pd:describe |
pd:round(2),@|
np:percentile(90),@|
sp:stats.norm.cdf,
sklearn:cluster.KMeans.fit_predict(3),
| @,| %,@| pivot(row;
column; value)
```

[live] reflected commands and library values: a library that reflects its package (`quarb_pyvalue::Reflective`: `quarb-lib-numpy` `np:`, `quarb-lib-pandas` `pd:`, and the undeclared names of `sp:` and `sklearn:`) resolves a dotted name through the package when the gate first meets it (`CommandLibrary::resolve`), the synopsis built from `inspect.signature` and leaked for the process: required positionals are operands, defaulted parameters options, `*args` the operand `...`, `**kwargs` the option `*` that opens the gate; a method's `self` is the input; a class on the path is constructed with the options its constructor declares, an intermediate object (`groupby`, `rolling`) takes its method, an attribute of a result is read during the run. The input is constructed once (a frame with the first text field as index, or the matrix of numeric fields) and a result that is not plain data is `Value::Foreign` (the `Foreign` trait: kind, display, rows, columns; `PyForeign` holds the object), equal only to itself, handed back whole to the next library command, printed by its own `str()`, exported by its rows in every output form and by the stdlib commands `@` (rows) and `%` (columns), refused by kind elsewhere; `transpose`, `pivot`, `melt` are the stdlib's reshapes

```
= #(@(name; age);
%("Braund"; 22)),
#(%(1; 2); %(3; 4)),
@| #, | :age, $_:shape,
fit((^/* | ::label))
```

[live] the table kind and the operand rule: `Value::Table` (`quarb::Table`: an optional header and rows of one width, `null` the hole; a frame with a header, an array without) written `#(...)` (`Token::Hash`, `Operand::Table`, the laws of the literal in `exec::operand::table_literal`), printed as its literal, read by `@`/`rows` (each row as the table has it) and `%`/`columns` (a record of lists, keyed by the header or by position), made by `#` (the table reading of a stream, `stdlib::table_reading`), a row by `| [n]` and a slice by `| [a..b]` (the plain pipe's positional bracket now parses and refuses at the run on anything but a table), `header` and `dim` beside them, a column by the value accessor (`:2` lexes for the array face), printed on the result line with each row on its own line, exported by its rows, crossing into a library command as its rows; `Operand::Index` (`v[2]`, `t[2; "age"]`, ranges in either position; `stdlib::index_value` behind it, `| [n]` and `cell(i; j)`), `row-to-header` and `header-to-row`; `pretty` the renderer (a `rendering` registry spec: a closing aggregate the tool takes off, and a scalar command after a value; `stdlib::pretty_value`, `tabulate_table`; `table` and `--table` aliases through 0.37); the trait `tabula` on every substrate's table (the text model's lowered table, renamed from `table`; HTML and XML `table` elements) and `exec::tabula::node_table` behind `| #` on such a node (lemma-keyed frame for the text model, declared header rows and spans for a grid); a DataFrame or a two-dimensional ndarray coming back is a table (the index the first column unless positional, NaN/NA a hole), a one-dimensional array or a positional Series a list (`quarb_pyvalue::foreign::as_table`). An operand of a library command keeps its kind: every positional argument of a reflected call goes through the same construction as the input, the input fills the first parameter named as data (`InputSlot`, `OperandOrSelf` for a class-rooted path whose instance on the pipe is the receiver), a bare constructor without a data parameter is built from its options alone

`| [::x =~ /(\w+)/] | %(k = $1), quai`	**[live]** match captures (filter-command `=~` binds groups onto the capsa) and the interactive session (living-query lines with rollback, session defs, `:show`/`:undo`; retired from `qua` in favor of the standalone `quai` crate; `NAME=TARGET` mount aliases via `quarb_session::MountSpec` and in-session `:mount` via `Session::set_executor` — in-process mode only; the Python `Session`/kernel accrete defs-only cells through the `parse_defs` binding)
`@| window(-2..0) | mean, @| window(..0) | sum, @| shift(1, ::k)`	**[live]** window functions: offset-range spans over context neighbors as members (partial edges; `window(n)` = trailing count; open start = expanding), `shift` lag with own registers kept, and optional partition keys (per-group windows in original row order)
`"${::name} (${::age})", macro &m($p, @rest): body;`	**[live]** string interpolation (double-quoted `${expr}` holes, evaluated in the current scope; null splices empty) and procedural macros: the body runs at expansion time against the invocation's expansion arbor (argument forms as reflected subtrees, `::form` = each form's text), literal args bind by value, `@rest` collects the remainder, and the text results — joined, reparsed against strictly-earlier fragments — become the expansion (`--expand` = macroexpand)
`macro &pivot!($col): /data/row | $col @| unique | ...`	**[live]** data-aware macros: `!` on the definition mounts the queried dataset as `/data` in the expansion arbor, and every invocation spells the `!` (enforced both ways); outside a hole a parameter is the argument form (call-by-name), inside a hole its text — so a pivot reads distinct data values at expansion and generates a column per value; `--expand` with an input expands against it
`| .(//*.rs @| count), | .t(^/row @| count)`	**[live]** subcontexts (grouped aggregation); a `^`-anchored branch navigates from the arbor root, so a subcontext can consult the whole table (share-of-total columns)

`.c(^/row[::k = $$.k] @\| count), \| %($$1/name::; /amt::)`	**[live]** correlated subqueries: the capsa is shared, so `$.name`, `$_` and `$ord` read the same register inside a body; `$$__` is the invoking position (`$$__::prop`, `$$__/child::x`), one query body out and no further, `$$_` the position and `$$0` the driver of a join condition (2026-10-18; `_` retired) — wide crosstabs, groupwise transforms; and the correlation keeps its *witness*, so `$$k` in pipeline commands projects the joined side into output (`SELECT a.x, b.y`); the witness travels into every path a command's operand or pipe hop navigates, so a nested step predicate (`((^/c/*[::b_id = $$1::id])`) reads the same thread's binding rather than re-searching the trace (2026-08-18)
`A <=> B[/k:: = $$/k:: and …]`	**[live]** correlation, *driver-first* (2026-07-25): the first expression drives (SQL's `FROM`); each joined expression carries its ON clause in its final step's brackets, reaching the driver as `$$0`, earlier entries as `$$k` and its own candidate bare or as `$$_` (the star spellings `$*` and `$*k` are retired, 2026-11-11: a node is `$$`, a value `$`); `<=>?` marks the expression it precedes nullable (`LEFT JOIN`); one tuple binds the whole predicate set, searched depth-first with per-entry pruning, and `$$k` may navigate from its context node (`$$1/field::`), so joins work where keys are children, e.g. JSON. Misplaced correlated references are parse errors, never silent nulls
`::ref~>`, `::'$ref'~>/city`	**[live]** cross-reference resolution (JSON: follow a `$ref` JSON Pointer)
`<--(::ref), <--(/scope/*::ref)`	**[live]** reverse resolution (2026-10-05): every node whose `ref` resolves to this one — a declared reference answers from the adapter's index, a scope bounds the search, and the whole-arbor walk runs only where the adapter opts in (`walks_for_references`), else the query is refused; `::a=>(/scope/*::b)` is the value match

`$.r:%provenance:text`, `:%provenance`, `:@origin`, `--reproducible`	**[live]** value provenance (2026-09-06/07): every value carries its *origins* (node, command; `Origins` bounded at eight, the rest counted) and the provenance entries of the nodes it was read from, kept in lockstep with record fields and list items (`Prov` in `pipeline/prov.rs`, thread-local read frames). A value's provenance reads through the shape sigils only (draft 157, `Operand::ValueMeta`): `:%provenance` the record (source, path, instant, instant-from, dpid, elided, text — the text being the kaiv canonical list), `:@provenance` one record per entry, `:%origin` / `:@origin` the node ids with the command; the bare keys (`value:::source`) refuse with a pointer to `:%provenance:source`, and on a node the same keys are the core rung (`/x:source`, `/x:%provenance`). `resolved_provenance` fills the name-path within the source and completes the instant ladder node → source (mount target mtime) → access (invocation instant); `--reproducible` holds the two lower rungs back. `qua --kaiv` exports per field and per plural on the semicolon with `;+N` elision (`--kaiv-origins`)
`.:name(body)`, `X:.name`, `:.name`, `X:%.`	**[live]** attached regulae (2026-10-31; the location-based cross-capsa read of 2026-09-07 retired): `Command::Attach` wraps the push, which files as itself, then attaches the regula it filed to each thread's node in the run's store (`pipeline/attached.rs`, keyed by `Attachment::Key`, type-erased in `RunState`); the store commits when the command ends, so reads see the previous command's attachments (the snapshot rule) and threads at one node attaching differing values refuse naming `@| collapse`. `Operand::PeerReg` reads by the node's identity (`at` optional: the current node), wherever a run is in progress — predicates included; a quantified push's fixpoint counts the node's attachments; `@| nodes` adds them as columns

`@| graph:pagerank(hop; --damping --rounds --weight), graph:distance(hop; --from --weight), graph:kcore(hop)`	**[live]** the graph library (2026-10-31; spec: The graph library): the engine's own command library, `graph/library.rs` (prefix `graph`, synopses with `InputSlot::Context`, answered by `RunOptions::library` for every build), every command a row-aligned aggregate: `graph_library_command` folds the threads to their distinct positions, `Host::graph_values` answers one value per position over the induced subgraph (`edges.rs`: `pagerank_values`, `distance_values`, `kcore_values`, the edge list folded by the hop as `edges` collects it), and each thread keeps its place with its position's value as the topic. The algorithms are the crate `quarb-lib-graph` (pure functions over index triples; the web level's stored rank calls the same `pagerank`). `--weight` has two forms on `edges` and the library (`Weight::Unit`, `Weight::Property`); `--from` is resolved from the arbor's root with the shared register; a library command never pushes — `| .:pr` and `| .:%` do (`Command::Attach` with `name: None` attaches every regula the fields push filed). New trait method `AstAdapter::edge_rows` (default `link_rows`; in the forwarding macro): the edges `edges` collects, which the web level reports page to page where a fragment link lands navigation on a section. The bare `@| pagerank` table of 0.44 is retired (its pointer names `graph:pagerank`)

`graph:artpoints(hop)`, `graph:bridges(hop)`, `graph:route(hop; --from --weight)`, `graph:betweenness(hop; --edges)`, `graph SCOPE(hop) { measure; }`

[live] the graph library's second shipment (2026-10-31; spec draft.189: The graph library, Graph Declarations, Other bounds): `GraphTable::{Artpoints, Bridges, Route}` and `Betweenness.edges`; a command that answers a table is a reducing aggregate (`GraphTable::answers_table`: `bridges_table`, `edge_betweenness_table` over `edges_table_of`, the edges table kept to a subset or with one more column), `artpoints_values` and `route_values` (lists of node references) per node. The algorithms are `quarb-lib-graph`: Tarjan's low-link over `Edges::symmetric` (the undirected reading), edge betweenness from the shared Brandes pass, routes with the predecessor of smaller index on a tie. The cost: before a library command computes, `Host::graph_size` counts the induced subgraph and the sum (the product for betweenness) against the graph bound (`AstAdapter::graph_bound`, ten million; `Run::graph_bound` / `qua --graph-bound`), refusing as `GRAPH_BOUND`. The declared graph: `AstAdapter::graph_declarations` (quarb-model's `graph` statements, `GraphDecl`; the web level's own `//page(->link) { pagerank; }`), `graph_measure` (the source's answer, `None` declines) and `store_graph_measure` (the engine's values offered back; quarb-model keeps them per mount), all in the forwarding macro. A mount is transparent to a declared graph (2026-11-01): quarb-mount lists every open mount's declarations with the scope prefixed by the mount's path (`/after//page`) and forwards the measure calls with the nodes translated to the mount's ids; quarb-model lists the base's declarations after its own and forwards the measure calls for a scope it does not redeclare, so a model's `mount` reads the site's stored column and `--diagnostics=info` says `PUSHED (graph)` there too. `declared_measure` matches a command to a declaration by the command's fingerprint (hop, weight and options, the declared measure parsed through the engine's own parser) and by the scope test (the declared path read as a node operand from the root equals the context's positions as a set), and reports

```
edges(->link[::lang
= "en"]),
graph:conncomp(--*[$-::src
> d]),:source::code,
map(::code)
```

[live] a predicate on a hop operand and reading through a reference (2026-11-01; spec draft.191): a hop operand takes exactly the predicate (and traits) a walk's hop takes, kept on `Hop::filter` and applied in `collect_rows` through the walk's own `apply_predicates` (the node arrived at the topic, `$-` the edge, `trait_clauses_ok` for traits), in `edges`, every library command and the declared graph's hop, whose fingerprint now carries the step; the `--label` special case of the argument parser reads the traits and predicates after the name. The graph methods of `Host` take the trace for it. `Operand::Through` (`REF::prop`, the postfix `::` after any operand) and the core-field read `REF:name` read the node a value names through `Host::ext_at` (`through` in `operand.rs`): a node reference, or each item of a list; a bare projection whose topic is a node reference reads it too, so `map(::code)` over a route and `(:source | ::code)` read the named nodes. Attachments read through as before

`graph:closeness(hop; --harmonic)`, `graph:communities(hop; --resolution)`, `graph:modularity(hop; --by=:.c)`, `graph:triangles(hop; --coefficient)`	**[live]** the graph library's third shipment (2026-11-01; spec draft.191): closeness in Wasserman and Faust's form or the harmonic (`quarb-lib-graph::closeness` over `distance` per source, in the hop's direction), the Leiden communities (`communities`: deterministic local moving in index order with ties to the earlier community, refinement from singletons, aggregation until nothing moves, over the undirected weighted adjacency), modularity of a given partition (`modularity`: Newman and Girvan over an undirected fold, Leicht and Newman over a directed one, the expectation scaled by the resolution), triangles and the clustering coefficient. `graph:modularity` is the library's first reducing aggregate that answers a value, not a table: `GraphTable::answers_one`; the executor reads the `--by` operand on every thread (`operand_scalar`), numbers the parts by first appearance, a null its own part, and `Host::graph_reduce` answers one float for the context. The inventory carries the four rows (lang 4a1b01db)
`AstAdapter::may_hold_below`	**[live]** a pruning hint for the deep walk (2026-11-01, cost only): whether a walk for a name and trait set may still find a match below a node, true by default; the engine's `descendants` asks it before descending when the step names a literal or carries un-negated, wildcard-free trait alternatives. The web level answers false below a page, a redirect, a resource or any document node when the walk seeks a row kind (page, dir, redirect, tag, category, resource) or a page-level trait (`tag:X`, `category:X`, `orphan`), so `//<page>` scans the page tree, not every page's document; mounts and models forward it

`?autounits`, `?autounit=_km,_g`; `quarb::autounits::AutoUnits`

[live] the unit suffix convention on demand (ruling 2026-11-08, the 0.57 round; spec: Mounts, the autounits option): on any target, every property whose last `_` segment names a unit the run knows (the built-ins, the GOST designations, a model's `unit` declarations, as spaced unit text — no opaque unit, a spelling the grammar refuses such as `mb` skipped) is read as the property without the suffix, holding quantities in that unit: a numeric cell or numeric text minted, a quantity of the dimension kept with its base and written in the unit, null null, any other cell as before; the named form converts the named suffixes only, stop list or not. `_at` and `_on` are temporal: the column typed in place (the name kept) as instants and as dates, a text cell by the ISO reading, a number by the one epoch rule (seconds; milliseconds from 10^{11} up — `Value::from_epoch`, shared with `datetime` and the `<instant>` cast). Under the flag a stop list of English words (`at`, `in`, `a`, `to`, `on`, ...) is never read as a unit, and a column converts only when the deciding row's cell admits the reading. `AutoUnits<A>` keeps the inner's node ids and forwards every reading through the macro; it decides a plan per header (remembered under the rows' parent) or per node's names, refusing a collision (a stripped name the rows carry, two columns stripping to one) and telling the conversions under `--info` (code `AUTOUNITS`, once per header). The property view and the child view agree: `property`, `properties`, `property_names`, `row_columns` and `row_text` (a numeric cell respelled `12_g` so the one-pass typed build types the column), and for a record's member nodes `name`, `children_named`, `answers_to`, `default_value`; a step plan naming a converted column — a renamed member as the step's name, a predicate or an order on one — returns `None` from `select`, `select_plan`, `estimate`, `count` and `index_lookup`, so the engine evaluates it; descendant searches by name are the engine's walk. quarb-open takes the options off the target before the source opens and spells a renamed member's locator by its new name; `qua` routes a target carrying them through the mount door as it does `?root=`. Tests:

`property SCOPE::name<TRAIT> = EXPR;` (a model file)	**[live]** a declared property (ruling 2026-11-08, the 0.57 round; spec: Model Files, the property statement): on every node of the scope — a path of child steps, with predicates — a property NAME whose value is the value expression at the node, the `::prop` reads the node's own, typed by the trait on the name: a unit `<_km>` as the typed weight clause types (a bare number or numeric text minted, a quantity of the dimension kept and written in the unit, another dimension or a non-number refused), a type word as the declared column casts; a name the node carries is retyped in its place (the bare form `property /legs::distance_km<_km>;` is `= ::distance_km`), a new name is added. Engine entry points: `parse_value_expr` / `ValueExpr`, `node_value` (the sibling of `node_matches_step_body`: `eval_operand` with the thread at the node, a default trace, no scope), `ScopeSteps` and `node_in_scope` (the steps matched against the ancestor chain), `cast_value`. quarb-model prepares the declarations once, answers them in `property`, `properties`, `property_names`, `row_columns`, `row_text` (a typed cell spelled in its type keeps the one-pass build; an untyped one steps to the record path) and `default_value` (a member node typed in place), hides the declaration evaluating at the node so its expression reads the source, refuses a cycle by name, declines the pushdown hooks on a declared name, and checks at load (`check_properties`: the unit known; the first node whose expression is not null typed). Tests: `qua/tests/model_property.rs`.

`//table[1] | #`, `#(--null=…)`, `#(--locale=…)`, `-i=null`; `quarb::graph::tabula`

[live] a document's table read by the constructor (the document-tables round, rulings 2026-11-08; spec: The Table, the constructor on a node): a tabula node — or its `thead`, `tbody`, `tfoot`, read as the rows of that section under the table's header — gives its body rows under the header the format declares (a first row of head cells, a `thead`, spans repeated), footer rows kept out of the body, a nested table refused by its place (*a nested table at row 2, column 2 …*). Every cell is handed to the one-pass typed build as text (`TableBuilder::push_text_row`), so a column of numbers types and a column that stays text keeps each cell as written — no more `5000000` beside `5,200,000[1]`; a footnote mark in a `sup` child is dropped from the cell's text, a trailing `[n]` too, and a `br` reads as a space in the HTML adapter's text (Markdown lowers through it). The reading options: `--null=@("—"; "n/a")` names the cell texts read as null (a document table's default vocabulary: the dashes, n/a, N/A, NA; `@()` none; a stream table has none unless given, so a `-` in a CSV stays text), `--locale="de"` the number convention (numeric cells respelled canonical, as `?decimal=` does); on a table value both pass over the cells and the columns retype. When every body row opens with a head cell, that column is the index, named by its label (`indexed_by`); `-i` overrides, `-i=null` keeps it a column. The table options (`-i`, `-c`, `--header`, `-a`) apply on the node as after `| #`; `--width` is refused there. A column that falls to text while a cell of it nearly reads as a number — a null word, a footnote mark, a percent, a scale suffix, another locale's spelling, a quantity — is told as a warning (`TABLE_TEXT`, site the table's path; up to three cells with their rows and reasons, then the cleanup). The aggregate path of `#` routes one table node through `Host::node_table(ext, &ReadOptions)` and `Host::is_table_node`; several nodes keep the record path. Tests: `qua/tests/document_tables.rs`.

`clean, clean(:a; :b; --null=…; --locale=…)`	**[live]** a scraped table's text columns cleaned into values (the document-tables round, hash 2, 2026-11-08): per cell, in the order a hand would — trimmed, a footnote mark dropped (`[1]`, `*`, †), a null word null (the document table's default vocabulary, or `--null=list`; `@()` none), the number read under `--locale`, a percent the ratio (`12 %` → 0.12), a scale suffix applied (`330.5 k` → 330500; M, G, bn, mn), else the text as trimmed — every text column or the columns named bare, the cleaned columns retyped (`Table::typed`, a declared column keeping its declaration), the index and the name kept; a text or a list of texts the same way (`clean_text`). The column names ride unevaluated, as `autocast`'s do. Tests: `qua/tests/document_tables.rs`.

`AstAdapter::surface`; `UNKNOWN_LABEL`, `UNKNOWN_PROPERTY`

[live] a name the surface a source declares lacks is told (the user's ruling of 2026-11-10: a misspelled label or property was silent). An adapter that declares every property its members have and every label that leaves or reaches them answers `surface(node)` for a member `node`: `Surface { container, properties, labels }`, each list whole or `None` where that side is open. quarb-relational answers, for a row, its table's columns and the labels of its keys and of the keys that point at it; quarb-csv, for a row, the header, the labels open; quarb-model the base's properties with its own declared ones, the labels open (a model's edge is labeled as it declares); the mount and compose forward to the adapter that owns the node and name the container from their own root (`Surface::under`: `/db/employees`), the forwarding macro to its inner one; the autounits wrapper stays open, its names being the renamed ones. The default is `None`, a source that declares nothing (JSON, a document store, the filesystem), of which nothing is said; so is a node that is no member (the table itself, a cell). The word is "declared", not "closed": a closed surface is the other thing, an adapter whose properties document content cannot grow. Two places ask, through `steps::unknown_property` and `unknown_label`: the one projection, for a projection and a predicate that read a property null (`project`), and a link hop whose named label no link of the node carries (`link_edges`; the navop wildcard is not asked, its name being a child's or the parent's as well). A warning, so it prints by default and the exit stays 0; once per declaring container and name (`ambient::surface_asked`; a source that declares nothing is not recorded, its containers being as many as its nodes), the names there listed, the nearest first where they are many. No answer changes. Tests: `qua/tests/surface.rs`

`agreed_keys`; `INDEX_CAST`, `TRANSPOSE_CAST`

[live] two indexes that meet agree on a type, level by level, before any label is compared (the user's rulings of 2026-11-10): a declared index wins and the other side's labels are cast to it (a label that does not cast refuses naming it), two undeclared are upcast to the least type every label of the two reads as, two declared are the same type or an integer with a float, which meet as floats, any other pair refused naming both. One helper, `stdlib::agreed_keys(command, t, u, pairs)`, asked by every aligning form before it matches: the append by label and the rows under one header (`++`, `++!`), an operator between two indexed tables, `correlate`. It answers the two tables with their key columns cast (`Table::with_column`), or nothing where nothing changes, so a caller calls itself again on what it answers. Where labels are cast it tells so, an info `INDEX_CAST` naming the command, the two indexes, their types and what they became; a transposition that brings the cells of columns of several kinds to one tells `TRANSPOSE_CAST` (`transpose_cast_told`). Both once per run for one text (`ambient::record_info_once`). Tests: `qua/tests/review_1110.rs`

`Table::levels`; `unstack`, `transpose`, `stack`, `t:bar`

[live] a colon in a source's column title is one name until a command makes levels (ruling 2026-11-10; any colon was a level, so the title `Time (HH:MM)` nested every row under `Time (HH)`. The levels are a mark on the table, `Table::levels`: false on every table read from a source or built from records (the builder, `Table::typed`, `Table::new`, `Table::assemble`); set by `unstack` over several data columns (the paths `column:value`), by the transpose of a composite index (`label_name` joins the levels) and by a literal whose header writes a path, `#(@(bar:A; bar:B; baz); …)` (`header_names` reads the segments glued by colons, a word or a quoted name each; `Operand::Table` carries `levels`); a quoted name alone is one name. The printers write a levelled column as its path and a name with a colon quoted (`header_spelling`, in `to_quarb` and the unparser), so both read back. The mark is one per table: in a header with levels every colon is a level, and the three makers refuse a colon inside a segment by name (the literal, a path beside a quoted name that holds one; `unstack`, a title or a label; the transpose, a label). Read by `header_has_levels` (the row and column readings nest by level only then), by `sub_columns` (`t:bar`), by `stack` and by the transpose's split of the old names. Every rebuild from a table carries the mark with the index and the name (`with_levels` beside `with_index` and `with_name`): a new site that builds a table from another's header chains it, or the levels read as names from there on. Tests: `qua/tests/review_1110.rs` (`a_colon_in_a_sources_title_is_one_name`).

#(--autounits), #(--autounit=:col…), #(--slugs[=sep]); the same on clean; ?autounits, ?slugs on a target; quarb::autounits::title_plan

[live] the title rules (the header-units round, ruling 2026-11-09): a unit at the end of a column title — (u), [u], , u, or the last word, against the run's unit table with the suffix convention's stop list and deciding-cell guard — is the column's unit; the title loses it and the column is declared in it (Draught, m → :Draught<_m>), a % title reads the cells as ratios (12 → 0.12, the point shifted on the text), a , at title is no instant. --autounits reads every title, --autounit=:col… the named ones (a name the table lacks refuses); --slugs slugs the titles — lowercased, every run of characters that are neither letters nor digits the separator (_, or --slugs="-"), the ends trimmed, letters of any script kept — after the unit comes off when both are on; two titles landing on one name refuse ('Quay, m' and 'Quay (m)' would both become 'Quay'). One info line per table (code AUTOUNITS, site the table's): the conversions, then slugs: the renames, then what was skipped. The rules live once, in autounits::title_plan / TitlePlan (the new header, the declared units, the ratio respelling), called by the document-table build (tabula::build, the options on ReadOptions), by stdlib::title_pass for a table in hand (# and clean, after the null/locale pass, before the index), and by the mount wrapper AutoUnits (one meaning everywhere: ?autounits reads titles as well as suffixes, ?autounit=_m takes Draught (m); ?slugs the names). The aggregate path strips the read options off the call before the table command runs on the result (without_read_options), so the rules apply once. Notation: --slugs[=sep] is the bracketed optional value (OptSpec::bare: bare a switch, valued it carries it). Tests: qua/tests/document_tables.rs, qua/tests/autounits.rs, the module tests of autounits.rs and signature.rs.

`12_%`, `<_%>`, `convert(%)`; the dimension-one rule	**[live]** the percent unit (ruling 2026-11-09): a built-in unit of dimension one with factor 1/100 of the ratio, declared table-wide on quanto's table beside the packs (`quantity::with_percent`; the sign a name character, so `12_%` lexes, `12 %` reads as unit text, a `<_%>` trait and `-c=:col<_%>` cast). `12_%` holds 0.12 (base `1`) and prints as written; `| type` says quantity; a `%` title under `--autounits` declares the column `<_%>` with its cells minted (`Share, % → Share in %`; the interim ratio reading is gone), `clean` makes `12_%` of "12 %". THE DIMENSION-ONE RULE (`arith::quantital_arith`, `dimless_rule`), for every unit of dimension one — the radian, the steradian, the percent, a declared one: such a quantity computes with a bare number and with a dimensioned quantity as the number it is (`12_% * 50` → 6, `1 - 12_%` → 0.88, `2_rad * 3` → 6, `12_% * 50_EUR` → `6_EUR`: the partner's unit, its magnitude scaled; a quotient by it the same); two of one dimension-one unit add to that unit (`12_% + 3_%` → `15_%`); a product or quotient of two is a bare number. Ordering against a bare number in the base (`12_% > 0.1`); equality never (`value_eq`: a quantity and a bare number are not equal, dimension one included — before this round the ratio pairing equated them). `convert(%)` / `convert(rad)` dress a bare number again (the number the base magnitude: `0.12 | convert(%)` → `12_%`); `unit(%)` puts a number in the unit (`12 | unit(%)`), `magnitude` reads it back. Tests: `qua/tests/percent.rs`, and the `%` cases of `qua/tests/document_tables.rs` and `qua/tests/autounits.rs`.

`correlate(t; --on=:col?; --inner? --first-wins? --last-wins? --many?)`

[live] the foreign-key join (ruling 2026-11-09; `stdlib::correlate`): the topic's key — the column `--on` names, its own index when omitted — matched as `=` matches (a number by magnitude whatever its spelling, a quantity in its base, text as itself; null matches nothing) against the argument table's one-level index (`#(-i=:col)`); the answer the topic's rows in the topic's order with its index kept and the argument's data columns appended in the argument's order, their declarations with them, the argument's index not repeated. An unmatched row keeps nulls; `--inner` drops it. A title the topic already carries lands as `title_2` (`_3`, ...), told. A label repeated in the argument's index refuses by default (`'LJU' twice (rows 3 and 7)`); `--first-wins` / `--last-wins` take one row of it, `--many` every row, the topic's row and index repeated. One info line (`CORRELATE`, site the topic's name or "the table"): the counts (`12 rows, 10 matched, 2 unmatched kept with nulls`), `--many`'s row count, the repeats taken, the renames. Refusals: a missing `--on` column, an unindexed or non-table argument, a composite key ("one key column for now"), two of the three repeat options. Tests: `qua/tests/correlate.rs`.

```
Throughput, metric
tonnes per year;
--units=%(word = _u),
--unit:col=_u,
--null:col,
--locale:col; ?units=;
quarb::value::unit_words
```

[live] the word road of the header rule (the header-units round, ruling 2026-11-09): a title's tail that is no symbol reads as WORDS through one exact dictionary in the engine (`unit_words`: English and Russian forms, singular and plural, the GOST designations beside; the power words `square`/`cubic`/`кв.`/`куб.` and a digit exponent `ft3`; the scale words `thousand`, `million`/`mn`/`млн`, `billion`/`bn`, `trillion` and a sign's glued letter `$m`; the connective `per`//`в`; the fillers `of`, `in`). The tail grammar `[scale [of]] phrase {per phrase} [scale]` gives a unit expression and a cell factor: `metric tonnes per year` → `t/a`, `[millions of cubic feet per second]` → `ft^3/s` ×1e6, `EUR million` → EUR ×1e6, `тыс. тонн в год` → `t/a` ×1000. THE SCALE RULE: always a cell factor, never a prefix change (`| scale` re-prefixes on demand); told "cells ×1e6". The tails tried longest first: a bracket, the text after the first comma, the last k words of a comma-less title (`Area in square kilometres` → `km^2`); the symbol road keeps a table symbol's own spelling (`кг`), a dictionary word takes the word road (`year` → `a`). The engine declares `a`/`yr` (365.25 d) and the short ton `ton` beside `%`; "tons" alone is the short ton, told "(short tons; write tonnes for t)"; "pounds" alone is ambiguous and skipped with the pointer "write £ (sterling) or lb (the mass)"; a tail that nearly reads is skipped and told (`'yer' is no unit`). The reading `Reading::Scaled(unit, factor)` carries the factor through the document build (`TitlePlan::factors`), the in-hand pass and the mount wrapper. The same dictionary serves `convert(tonnes)`, `12_tonnes` (= `12_t`, printed as written), unit text in cells (`5 metric tonnes` → `5_t`, which `clean` lifts) — the resolver reads the strict table, then the words, then an opaque unit, so a currency sign names its currency (`12_€` = `12_EUR`). THE OVERRIDE: `--units=%(pounds = _lb; tons = _t)` maps words before the dictionary for every title, `--unit:col=%(…)` for one column, `--unit:col=_lb` declares the column in the unit whatever its title says (a unit-word tail dropped, told "(declared)"); a dictionary alone implies the reading of those titles; `?units=pounds: lb,tons: t` on the

`::mass @|` `sp:describe`, `np:mean`, `pd:…` over quantities; `quarb_pyvalue::dress`	**[live]** the unit dressing of a library call (the user's rule, 2026-11-09, after lang's finding on 0.57.0 that `sp:describe` over grams answered base kilograms as bare numbers): a sample of quantities of one dimension — the stream a Python library command receives, a list or a one-column table — computes on the magnitudes in the FIRST value's written unit, and the answer wears the unit again by the field's (or the command's) name: the moments that carry it (`mean`, `min`, `max`, `median`, `std`, `sem`, the quantiles, a sum) in the unit, a `variance`/`var` in the unit squared (`g^2`), the counts and the shape statistics (`nobs`, `skewness`, `kurtosis`, a test statistic, a p-value) bare. Two dimensions mixed, or quantities beside bare numbers, refuse by name ("quantities of unlike dimensions (kg and m) in one sample; convert first"). One place for every Python library: the glue (`PyGlue::call`) and the reflective road (`Reflective::call`) lift the sample through `dress::lift` before Python sees it and dress the answer through `Dress::apply`; a sample of bare numbers crosses untouched. Tests: the scipy crate's `a_quantity_sample_answers_in_its_written_unit`.

`quarb::rich`; `Session::eval_rich`; `QuaiSession.setPretty`; `Document.run_rich`	**[live]** the rendering rule (the user's ruling, 2026-11-09): one renderer for every browser host of what a result looks like as HTML, and one rule for WHEN — the session's `--pretty` setting. `rich::html_of(values, closing_grid, drawing, mode)`: under `off` nothing (the text alone); under `structures` a TABLE value as its grid drawn from the table itself (`rich::table_grid`: the index levels first, marked `class="index"`, the header a title with the type word beneath — the declared unit or the inferred word — cells through the pretty float; one grid per table, stacked in thread order), a record stream closed by `@| pretty` as the grid (the `run_table` road), a drawing (`@| tree`, `@| histogram`) as its HTML; under `always` a bare record as a two-column key/value grid and a list of scalars as one row across. The literal stays in the lines; a table inside a cell is its literal text. `rich::css()` is the one stylesheet (`TABLE_CSS` with the `.index`/`.type` rules and the diagnostics block's), `rich::diagnostics_html` the block under an answer (warnings as lines, infos behind a `<details>`), `rich::document` the three together for an export. The one-cell bug: `table_rows_of` expands a TABLE value among the rendered values into its rows' records, so `//table[1] | #N | clean @| pretty` draws the rows in both forms. The hosts differ in nothing but the setting they open with: quarb-session's `Executor::run_rich` / `Session::eval_rich` run every line through the rule (`set_pretty`, `pretty_mode`, `rich_of` for a frozen replay, `export_rich`); quai-wasm's `run`, `run_cell` and `&N#` fill the envelope's `html` from it, `setPretty(word)`/`pretty` set and read the setting, `richCss()` is the stylesheet, `render(line, "html")` the document form; the extension's worker opens at `structures`, the playground at `off` with a "grid" button that runs `session(--pretty=…)` as a line would; the Python `Document` carries a session memory opening at `structures` (so `session(--pretty=…)` works in a notebook), `run_rich` answers values, text and HTML, `quarb.rich_css()` and `quarb.diagnostics_html()` serve `repr_html`. Tests:

`| slug`, `slug("-")`	**[live]** the text as a slug (the header-units round, 2026-11-09): the same slug the `--slugs` option makes of a title — lowercased, non-alphanumeric runs the separator (_ by default, the argument otherwise), the ends trimmed; a list item by item; null where nothing is left (`autounits::slug_text`). Tests: `qua/tests/document_tables.rs`.
`?root=PATH`, `qua --root PATH`; `quarb::rooted::Rooted`	**[live]** a root on a mount (ruling 2026-11-01; spec: Mounts, the root option): the subtree at PATH mounted as the tree, on any source (`web:after.tgz?root=/sites/quarb.org`, a model's `mount after: …?root=…;`). `Rooted<A>` keeps the inner's node ids and forwards every reading; the root's parent is none, paths and provenance come back relative to the root (`/pages/index.html`), and the membership test (the parent chain reaches the root) gates what would reach out: links, backlinks, edge rows, resolutions, index lookups, selections and the non-tree nodes drop whatever lies above or beside the root, so a hop that would leave it answers nothing. A declared graph's scope is read inside it (the web level's `//page(->link)` still matches). quarb-open takes `root=` off any target before the source opens (`open_mount`) and renders the locator relative too; `qua --root PATH` is the sugar for one source. A PATH no node matches, or several, refuses at mount

| `| #(:a; :b)`, `| %(:a; :b)`; `round` / `floor` / `ceil` / `abs` | **[live]** rulings 2026-11-01: a `#(` or `%(` after a pipe whose items open on a projection is the projector applied to the topic, the pipe form of `t:#(:a; :b)` / `v:%(:a; :b)` (`Operand::Projected` over `Operand::Topic`; the parser's `projector_items_ahead`); a record shape over a list of records reads each item; the web planner keeps its prefilter rung under such a projector. The numeric commands (the rounders, `abs`, the logarithms, `exp`, `sqrt`, `pow`) leave a cell of a kind they do not take as it is wherever they broadcast, so a text column of a table survives `| round(3)` and `| sqrt`; a lone value of such a kind answers null, and the run warns once for the command, counting the values (`NOT_NUMERIC`; ruling 2026-11-10). The rule is decided in `apply_scalar` (`untaken`); the count rides the ambient (`record_untaken`), and `map` marks its body's commands as standing on a cell (`mark_cells`, taken by the nested pipeline's `cells_ground`). Beside them: a web `<dir>` row answers `::name`, and a page reference held in a value prints its arbor path (a root-only provenance path falls back to the name path) |

AstAdapter::property_withheld, withheld_properties, nodes_withheld; OMITTED

[live] omitted data (2026-11-01; spec: omitted data, draft.192): data a source withholds from the caller. Three hooks, all defaulting to silence: `property_withheld(node, name)` (asked after an absent read: the property stays absent, never null, and the run marks the omission), `withheld_properties()` (source-wide names the planner never pushes as a conjunct or an index lookup) and `nodes_withheld()` (asked once after the run: whole nodes dropped, simply not there). The report is one `warning OMITTED (source): some data is omitted due to insufficient permissions` per run (`Ambient::omitted`, pushed when the ambient closes, or by `Query::run` for nodes), never twice and never beside the error of the same code. The error: a withheld read inside a key evaluation (`Ambient::keying`, raised by `eval_pred_expr_at` and the filter command) refuses as `error OMITTED (source)` with the same text; a value that came from a withheld read carries `Origins::omitted` through the captures, and the order, rank (top/bottom), filter and value-combining aggregates refuse over a marked thread, while `count`, the structural commands and the renderers (`OMISSION_SAFE`) pass. Nothing is ever null for this reason

`$-:source`, `$-:target`, `:source:.l`	**[live]** edge ends (2026-10-31): the arrived-by edge's two ends as node references in the declaration's orientation (`Host::edge`, `CoreMeta`); the `@| edges` columns renamed `source` / `target` and, like `nodes`' `node`, node-valued (`node_ref`, printing as the path). A node reference is a node side of the node read: `operand_nodes` resolves `Operand::Edge` and `Operand::Field` through `ref_nodes`, so a row filter on the edges table reads both ends' attachments (`[:source:.l = :target:.l]`)
`((!7))`, `((!$.v:%origin:id))`	**[live]** the node-id anchor (2026-09-07): `Anchor::Id` takes an expression; a number, a `%origin` record (by `id`), a list forks. `AstAdapter::has_node` guards it (default: probe parent/name under `catch_unwind`, since most adapters index a Vec by id). Ids must be deterministic across runs — minted from the node's identity in the source, never from walk order
`| .v(body){20}`, `{1;}`, `{m;n}`	**[live]** the quantified push (2026-09-07): `Command::Repeat` wraps a push-family command (push, expression push, subcontext, record and fields push); each round is one `apply_command` over the whole context, so a round reads the previous round's snapshot; `{N}` exact, `{m;n}` / `{m;}` at least `m` then stop at a fixpoint (no node, topic, or `%.` view changed) or at `n` / the quantifier bound. PageRank over quarb.org is one such push; the stored column matches to 2×10^{-17}

The pipeline runs on *capsae* — the register system's unit, a node paired with a register (its breadcrumbs, node references among them) and a current topic. A per-capsa `|` transforms each topic; `@|` reduces the whole context; `| .name` pushes the topic onto the register (a reference to the position where there is no topic) and `| $.name` recalls it; a subcontext `| .(expr)` evaluates `expr` from each node, reduces it to a scalar, and pushes it — grouped aggregation without leaving the query. A push of any spelling appends to the capsa and leaves the cursor as it is (spec draft 154): the topic before `| .n(expr)` is the topic after it, the value waits in the regis-

ter for `$.n`, and a query ending in a push yields its topics as they stood (`pipeline_projected` treats no push as value-typing). The context family reads the command's input snapshot (spec draft 157): `@_` the peers' topics, `@@_` their positions (`Host::positions`; a glued projection maps over them), `#.*` and `##.*` the context as a table of the value or the node regulae (`pipeline::operand::matrix`, one row per thread, the columns as `%.*` names them, typed by the table rule; `#.` and `##.` the named regulae alone); the old `@*` refuses with the pointer. A push quantified `{N}` repeats over the whole context, each round seeing the last (the iteration loop); `X:.r` reads the register of whichever capsa stands at the node `X` reaches, from the same snapshot; and every value remembers where it was read, so `:` on a value answers its provenance the way `:` on a node does.

Any other operator is lexed and rejected with a `not yet supported` error naming the offending construct, rather than silently mis-parsed. Everything in Section 3.4 falls into this category today.

Chapter 3

The Adapter Surface

An *adapter* maps a data source onto the Quarb arbor model — a tree backbone whose edges carry *names*, enriched with non-hierarchical crosslinks. The engine drives all navigation through the `AstAdapter` trait, so this trait *is* the contract between the language and the world.

One clause of that contract concerns the values an adapter mints (spec: adapter responsibilities, draft 155): a property that *measures* something is a quantity in the source's own unit — a size through `Value::bytes`, a span of time through `Value::seconds`, a length as a `Value::Quantity` written in millimetres or kibibytes as the source keeps it — and a count stays a number. A bare number never compares with a quantity (a threshold names its unit, `[::size > 40_B]`), so a measure minted bare would be unreachable from a criterion.

A second clause concerns traits (spec: adapter traits, draft 156). A relational cell carries its column's declared type, lowercased and stripped of parameters (`<varchar>`, `<integer>`), and beside it the derived kind `<number>`, `<string>` or `<boolean>`; the one implementation is `quarb_relational::type_traits` over the `TableSpec::column_types` each driver fills from its catalog (PRAGMA `table_info`, `information_schema`, Oracle's `user_tab_columns`). A file carries its lowercased extension beside its class (`<json>`, `<tgz>`; `.tar.gz` carries `<gz>` and `<tar>`), so the leaf of the file system and the root of the grafted document say what they are alike.

3.1 The Live Trait

The methods the skeleton actually uses. Signatures are the real ones from `quarb/src/graph/adapter.rs`:

```
/// An opaque handle to a node, minted and interpreted by the
/// adapter that produced it.
pub struct NodeId(pub u64);
```

```rust
pub trait AstAdapter {
    /// The root node -- the initial navigation context.
    fn root(&self) -> NodeId;

    /// The tree children of `node`, in document order.
    fn children(&self, node: NodeId) -> Vec<NodeId>;

    /// The name of `node` -- the label of its incoming tree edge.
    /// `None` when the adapter leaves a node unnamed (e.g. the
    /// filesystem root).
    fn name(&self, node: NodeId) -> Option<String>;

    /// The parent of `node`, or `None` for the root.
    fn parent(&self, _node: NodeId) -> Option<NodeId> { None }

    // --- projection surface (defaulted; implement what fits) ---

    /// Adapter-defined classifications, for `<trait>` filters.
    fn traits(&self, _node: NodeId) -> Vec<String> { Vec::new() }

    /// A named property `::prop`.
    fn property(&self, _node: NodeId, _name: &str) -> Option<Value> {
    None }

    /// The default projection, bare `::` (filesystem: file content).
    fn default_value(&self, _node: NodeId) -> Option<Value> { None }

    /// Adapter metadata `:::key` (filesystem: size, modified, …).
    fn metadata(&self, _node: NodeId, _key: &str) -> Option<Value> {
    None }

    /// Outgoing / incoming crosslinks as (label, node) pairs, for
    /// `->` / `<-` (filesystem: symlinks).
    fn links(&self, _node: NodeId) -> Vec<(String, NodeId)> { Vec::new
    () }
    fn backlinks(&self, _node: NodeId) -> Vec<(String, NodeId)> { Vec::
    new() }

    /// The same with each edge's identity: the node that IS the
    /// edge where edges are rows (a model's `edge … as flight` over
    /// a flights table), `None` for a plain link. Two rows on one
    /// leg are two edges, each its own thread and its own `$-`; the
    /// row's cells are the edge's properties. Defaulted from the
    /// pair forms with no identity.
    fn link_rows(&self, _node: NodeId)
        -> Vec<(String, NodeId, Option<NodeId>)> { … }
    fn backlink_rows(&self, _node: NodeId)
        -> Vec<(String, NodeId, Option<NodeId>)> { … }

    /// Whether the links into `node` are ones this source cannot
    /// walk back: it answers `links` and provides no `backlinks`
    /// (a document store whose references live in the referring
    /// documents). The engine then refuses the hop that needs the
    /// backward direction (`<-`, `--`, the navop wildcard, `edges`
    /// over any of them) rather than answer an empty set that
    /// would read as "nothing points here". `label` is the one
    /// label the hop names, `None` for a wildcard, a pattern or a
```

```
    /// selection: a layer that walks some labels back itself (a
    /// model's declared references) answers for those and hands
    /// the rest down. A source with no links at all says no.
    fn backlinks_unsupported(&self, _node: NodeId, _label: Option<&str
    >)
        -> bool { false }

    /// Whether the link from `source` under `label` is a reference
    /// an edge declaration consumed: a bare wildcard hop (`->*`,
    /// `<-*`, `--*`, the crosslink part of `~`) skips it, the label
    /// named still walks it (spec: Edge rows). A model's `edge
    /// /flights/*: ::origin -- ::dest` consumes the two references
    /// its fields resolve through. Defaults to nothing consumed.
    fn link_consumed(&self, _source: NodeId, _label: &str) -> bool {
    false }

    // --- the reference surface (defaulted) ---

    /// (The name glued to a resolution arrow is its reference label;
    /// the parameter keeps its historical name `hint`; reflection
    /// reports it as `label`.)
    /// Resolve a cross-reference: `::prop-->label` maps a property
    /// value to its in-document target (html: a `#fragment` href
    /// to the element with that id).
    fn resolve(&self, _n: NodeId, _prop: &str, _hint: Option<&str>)
        -> Option<NodeId> { None }

    /// The external reference a property holds -- an
    /// adapter-defined identifier of a document outside the arbor
    /// (html: the absolute URL, relative hrefs joined against the
    /// document's declared URL). On an in-document miss the engine
    /// consults the mount layer's reference targets (identifier ->
    /// mounted root) and lands, or records the identifier among
    /// the run's unresolved external references for the host's
    /// acquisition loop. The engine performs no IO, ever.
    /// (document, location within): the adapter reads both out of
    /// its own reference syntax; the engine splits nothing.
    fn external_ref(&self, _n: NodeId, _prop: &str, _hint: Option<&str
    >)
        -> Option<ExternalRef> { None }

    /// The element `fragment` names inside `node`'s document --
    /// the landing rung of a fragment-carrying external reference.
    fn resolve_fragment(&self, _n: NodeId, _frag: &str)
        -> Option<NodeId> { None }

    /// The property carrying the node's own reference, for the
    /// bare arrow `//a-->` (html: an anchor's href); and the
    /// relation an edge carries (html: `rel`), the label falling
    /// back to the property name.
    fn ref_property(&self, _n: NodeId) -> Option<String> { None }
    fn ref_label(&self, _n: NodeId, _prop: &str) -> Option<String> {
    None }

    // --- the index hooks (defaulted; an indexed store answers) ---

    /// The document an external identifier names (a URL -> its
```

```
    /// page), the arrow's landing rung after the mount layer.
    fn document_by_ref(&self, _id: &str) -> Option<NodeId> { None }
    /// `<--(::prop)` from an index (a declared reference);
    /// `None` means no declaration answers.
    fn reverse_resolve(&self, _n: NodeId, _prop: Option<&str>,
        _hint: Option<&str>) -> Option<Vec<NodeId>> { None }
    /// Whether an unscoped `<--` may walk the whole arbor here
    /// (the in-memory formats say yes; a catalog says no).
    fn walks_for_references(&self) -> bool { false }
    /// `index SCOPE::prop;` (a model statement): the scoped nodes
    /// holding `value`, and those whose property resolves to a
    /// target, from an index the model keeps. Cost only: the
    /// caller still tests each node, so the answer is the walk's.
    fn index_lookup(&self, _scope: &str, _prop: &str, _v: &Value)
        -> Option<Vec<NodeId>> { None }
    fn index_referrers(&self, _scope: &str, _prop: &str,
        _label: Option<&str>, _target: NodeId)
        -> Option<Vec<NodeId>> { None }
    /// Can a reference *declared* on `source` (a catalog key, a
    /// model `ref`, a reference property the format defines) land
    /// in `target`'s container? Schema, never data; `None` where
    /// nothing is declared. Decides `<--(SCOPE::*)` with no label.
    fn can_refer_to(&self, source: NodeId, _target: NodeId)
        -> Option<bool> { self.ref_property(source).map(|_| true) }
    /// The reference properties declared on `source` into
    /// `target`'s container: what `t<--` and `t<--(SCOPE)` read.
    fn reference_properties(&self, source: NodeId, _target: NodeId)
        -> Vec<String> { self.ref_property(source).into_iter().collect
    () }
    /// Whether the source resolves references at all (a CSV file
    /// does not): both arrows are refused where it does not.
    fn resolves_references(&self) -> bool { true }
    /// The label a label-less reverse arrow also resolves under,
    /// when the question is about `target` (a database: its
    /// table). None by default; the engine derives none.
    fn reverse_label(&self, _target: NodeId) -> Option<String>
        { None }
    /// The trees other trees refer to by name: (identity, node);
    /// an identity ending in a separator is a base. Declared in a
    /// model (`tree IDENTITY: PATH;`), automatic for a mount's
    /// own target. The engine reads prefix and separator, no more.
    fn tree_identities(&self) -> Vec<(String, NodeId)> { vec![] }
    /// Whether `::prop-->` needs a reference label here (a column
    /// with no key, a property graph): refused without one.
    fn resolve_needs_label(&self, _n: NodeId, _prop: &str) -> bool
        { false }
    /// A node's property names, for `<--(SCOPE::*)`; an element
    /// lists its attributes.
    fn property_names(&self, _n: NodeId) -> Vec<String> { /* … */ }
    /// `//name` below `node` from an index, with each hit's depth;
    /// `None` means walk. The answer must equal the walk's.
    fn descendants_named(&self, _n: NodeId, _name: &str)
        -> Option<Vec<(NodeId, usize)>> { None }
    /// The search fast path: one step's plan (axis, name, the
    /// equality conjuncts the engine lowered, a limit, an order)
    /// answered from an index -- `None` walks. The default folds
    /// `children_named` / `descendants_named` / `descendants_where`.
```

```
    fn select(&self, _from: NodeId, _plan: &StepPlan<'_>)
        -> Option<Selection> { ... }
    /// The dry-run form of `select`: which conjuncts the adapter
    /// would apply, from its catalog alone (`scope` is the literal
    /// path from the root to the step's base); `None` cannot say.
    fn select_plan(&self, _scope: &[&str], _plan: &StepPlan<'_>)
        -> Option<Vec<bool>> { None }
    /// What reaching the step would cost, from the catalog: row and
    /// byte intervals, requests, a latency class, a confidence
    /// (a dry run, statistics, a file's size, an arena's count).
    fn estimate(&self, _scope: &[&str], _plan: &StepPlan<'_>)
        -> Option<Estimate> { None }
    /// How many nodes the step reaches, without making them:
    /// `@| count` right after a bare step asks (SQL's COUNT(*), an
    /// index bucket's length). `None` walks.
    fn count(&self, _from: NodeId, _plan: &StepPlan<'_>)
        -> Option<usize> { None }
    /// The columns of the rows `row_text` serves, for the source
    /// `node` belongs to: a source whose rows are records of one
    /// shape (a CSV's) names them; `None` where they have no one shape
    .
    fn row_columns(&self, _node: NodeId) -> Option<Vec<String>> { None
    }
    /// The surface the source declares for the container `node` is
    /// a member of (ruling 2026-11-10): the properties its members
    /// have and the labels that leave or reach them, where the
    /// source declares them all (a table's columns and keys, a
    /// CSV's header). `None` is a source that declares nothing.
    fn surface(&self, _node: NodeId) -> Option<Surface> { None }
    /// The node `node` stands for (identity is intrinsic, ruling
    /// 2026-11-06): a model's role node answers the source it aliases.
    fn intrinsic(&self, node: NodeId) -> NodeId { node }
    /// A row node's cells as text in the columns' order, handed to
    /// `f` once (an empty text: null) and `true`; `false` where the
    /// node is no row of that shape. The one-pass typed table build
    /// parses the cells from the text.
    fn row_text(&self, _node: NodeId, _f: &mut dyn FnMut(&[&str]))
        -> bool { false }
    /// A batching hint before the engine reads the properties
    /// `names` of every node in `nodes` (the next command projects
    /// them): the projection twin of `prefetch_links`.
    fn prefetch_properties(&self, _nodes: &[NodeId], _names: &[&str])
    {}
    /// A batching hint before a link sweep over `nodes`.
    fn prefetch_links(&self, _nodes: &[NodeId], _dir: LinkDir) {}
    /// The weights an edge carries (spec: the edge model, ruling
    /// 2026-11-07): quantities of pairwise distinct dimensions, none
    /// dimensionless, in the canonical dimension order -- what a
    /// budget sums for its dimension and `--weight=UNIT` reads,
    /// never inferred from the properties. A model declares them
    /// (`weights(::km)`); a substrate with weights of its own mints
    /// them. `row` is the edge's own row where the edge is one.
    fn edge_weights(&self, _s: NodeId, _label: &str, _t: NodeId,
        _row: Option<NodeId>, _forward: bool) -> Vec<Value> { Vec::new
    () }

    // --- identity and provenance (defaulted) ---
```

```
    /// Whether `node` is a node of this arbor -- the guard of the
    /// node-id anchor `((!expr))`. Default: probe parent and name
    /// under catch_unwind.
    fn has_node(&self, node: NodeId) -> bool { ... }
    /// What the source recorded about `node`: source, path within
    /// it, instant (and which rung it came from), dpid; a list when
    /// several sources stand behind one node (mounts, grafts,
    /// models layer entry by entry).
    fn provenance(&self, _n: NodeId) -> Provenance { ... }
    fn provenance_list(&self, n: NodeId) -> ProvenanceList { ... }
}
```

The navigation methods are required; the projection methods carry defaults, so an adapter implements only what its domain offers. Core metadata (`:key` — index, depth, counts, `is-leaf`, ...) is computed by the engine from the navigation surface, so no adapter implements it.

Three observations that the filesystem work has already forced:

- **Names live on edges, not nodes.** `name` returns the label of a node's *incoming* edge — its role in its parent — exactly as the specification's edge-name model requires. For the filesystem this is simply the filename.
- **`NodeId` is opaque and adapter-private.** The engine never inspects the `u64`; it only passes ids back to the adapter. Each adapter chooses its own identity scheme (the filesystem adapter uses an interning arena index).
- **New trait methods need their forwarders.** The run path wraps adapters in layers (the decoding `Sourced` layer), the session in `Dyn`, mounts in `MountAdapter`/`Shared`, models in `ModelAdapter` – a defaulted method silently answers `None` through any wrapper that does not forward it. The plain wrappers (those that override at most a method or two) write their forwarding with `quarb::forward_ast_adapter!(FIELD; except m, …)`, whose method table is checked against the trait by a test in `adapter.rs`, and a probe adapter is run through each shipped wrapper – so a new trait method is forwarded by all of them once it is added to the table. The composing wrappers (`MountAdapter`, `ModelAdapter`, the graft, the reflection views) still forward by hand: when adding to the trait, sweep those. The case in point (2026-09-07): `Dyn` had never forwarded `answers_to`, so a web level behind the session's boxed door answered `//page` with nothing – its rows answer to their kind through that method. The sweep of 2026-10-06 found three hooks no wrapper forwarded (`descendants_where`, `has_node`, `provenance_list`), and `ModelAdapter` dropping `link_property` and aliased metadata, so an edge property or `:::size` answered nothing under `--model`; the macro round of 2026-10-07 found the Python binding's `Dyn` forwarding 29 methods of 45. A model's mount targets stay relative to its own file (the tool resolves them once, against the

file it was given), so a `used` model's targets are carried over by the `use` path as written.

- **Integers compare as integers.** Two integers are compared, keyed for joins and `unique`, and sorted exactly; the float reading enters only when one side is not an integer (`SortKey::Line` carries the integer beside the float, and -0.0 keys as 0.0). Above 2^{53} the float reading alone would merge neighbours.

- **The core is smaller than the crate.** The temporal fragment (instants, durations, periods, time expressions, the calendar names) is the `quarb-time` crate, re-exported by `quarb` at its old paths (`quarb::temporal`, `period`, `timex`). Three features, on by default, carry what one function needs: `decode-formats` (the structured readings of `decode`: serde_json, serde_yaml_ng, toml, quick-xml), `kaiv` (the kaiv output of `| kaiv` / `@| kaiv`), `desm` (syndesmos as the session's sentence bonds). Off, the function refuses with the feature's name. The sentence bonds themselves are a trait (`quarb::SentenceBonds`): the engine segments through whatever the tool set, syndesmos or another.

- **The plan is read before the run.** `Query::plan(&adapter)` (`quarb::plan`) reads off the syntax tree, per branch and per step, the axis the fast path covers, the literal name, the equality conjuncts lowered, the literal path to the step's base where every step before it was a bare child hop by name, and what the adapter says from its catalog: `select_plan` (which conjuncts it would apply) and `estimate` (nodes, bytes, requests, latency, confidence), composed along a branch (`Estimate::then`) and across branches (`beside`), unknown where no estimator answers. `Plan::roots` names the first child of every anchored branch – under a mount table, the mounts the query touches; an unanchored search is every mount. A `Run` may carry a `Budget` (nodes, bytes, requests; `strict` refuses an unknown estimate): a query the estimate puts beyond it is refused before anything runs, and the `Outcome` carries the plan. `Plan::to_kaiv` emits it as a `quarb/plan` payload (`qua --plan` prints it and stops; `qua --budget nodes=N,bytes=N`). `Model::reachable_roots(nam` (quarb-model) closes a set of mounts over the `ref`, `rel` and `tree` declarations, for a replay plan. The relational model takes a driver's *estimator* (`RelationalModel::with_estimator`: the table and the plan's column equalities in, an `Estimate` out, asked once per run per table and equalities): BigQuery prices the statement with a dry run (bytes exact, a bound for a clustered table) and reads the row count from `tables.get`; Postgres, MySQL and Oracle ask their planners (`EXPLAIN`, `EXPLAIN FORMAT=JSON`, `EXPLAIN PLAN`); SQL Server reads `sys.partitions`; DuckDB its catalog's `estimated_size`; Athena the table's crawler properties; SQLite `sqlite_stat1` after `ANALYZE`, unknown without it. The SQL adapters forward the whole trait through

`forward_ast_adapter!`, so `select`, `count`, `select_plan` and `estimate` reach the model.

- **A mount may be latent.** The mount adapter declares a mount by name, target and opener (`MountAdapter::declare`) and opens it the first time a query reaches it: a hop by its name, a listing of the root's children (so an unanchored `//x` opens every mount before the scope memo is built), a reference landing on a declared identity. `tree_identities` answers a latent mount from its declaration with a stand-in root id that opens the mount when touched; the every-mount hints read the active set. A failed opener is a refusal and an empty mount. A live table takes new mounts (`push`) and identities (`declare_identity`). A declaration may carry a `catalog` closure: what the target holds before it is opened, so the plan prices a step from the mount's root while the mount is latent.
- **A catalog prices a source before it is read.** `quarb::Catalog` is substrate-neutral — entries and bytes as a range, a latency, a confidence — and every substrate states one cheaply: `quarb_archive::catalog` from a zip's central directory or a tar's headers, `quarb_open::catalog` from a bounded directory listing or a file's size, a warehouse from its dry run. `Estimate::from_catalog` turns it into a root step's estimate as bounds (entries are files or members, the estimate counts nodes: a child step reaches at most the entries, an unfiltered descendant step at least every entry), the catalog is a lookup by literal path (`CatalogLookup`: a folder's own listing, one member's header), so `Estimate::at_catalog` narrows the price to the path the plan records — a bare hop by name is one entry, the reading step under it is priced by that entry's catalog, a path the catalog cannot place (a node inside a document) is a bounded unknown under the whole; a mount table prices a root step across its mounts; a database file is left to its store's estimator once open, and a hop by name under a table is a row by key, at most one. The plan lowers the leading predicate as a comparison (`Conjunct::Compare`: any operator, any literal; a text equality stays `Equals`, the index's) and the step's trait filters, and asks the catalog with them (`Ask`): a listing applies `::size` and `<file>`/`<dir>` before counting and says so (`Catalog::applied`), a store's statistics price an equality on an indexed column (SQLite: `sqlite_stat1`'s rows per key); what a catalog cannot apply leaves the whole count as the bound; `quarb::Unopened` is an adapter that is only its catalog, for planning and budgeting without opening (`qua --plan` / `--budget` price every path input this way first). Once open, an in-memory adapter answers `estimate` through `adapter::arena_estimate`, a counted walk of its own nodes (JSON, CSV, XML, HTML, text, fs, archive), bounded past two million nodes. An upper end of `u64::MAX` means unbounded: a lenient budget passes it, a strict one refuses. The budget's cap (`Budget::cap`, carried as `StepPlan::cap` and handed to every catalog producer)

short-circuits the pricing itself: a listing, a tar header walk or an arena count stops at the first entry past the budget and reports a lower bound that already exceeds it, so pricing a directory of a hundred thousand files under `nodes=1000` costs a thousand reads and one. Pricing is opt-in: a run with neither `plan` nor a budget never asks an adapter for an estimate and never reads a catalog, so the default path runs the query straight, at no planning cost.

- **The conformance suite is the contract's test.** `quarb::conformance::check(&adapter, budget)` walks a sample of an adapter's nodes and names every pair the engine relies on that the sample breaks: a child's `parent`, `has_node`, `answers_to` its own name, `children_named` and a child `select` finding it, `has_trait` over `traits`, `property_names` over `properties`, `provenance_list`'s first entry, a descendant `select` against the walk. An adapter's tests call it on a fixture and assert the list is empty (the JSON and HTML adapters do); an index that answers short is caught there, not by a query.

- **The shell gate is checked twice.** The static walk refuses a query that names `sh(...)` or a backtick command anywhere it can stand (record fields, list items and field bases included); the executor refuses the command again when it runs, so a path the walk does not know cannot reach the shell without `--allow-shell`.

- **A child's standard input is fed from a thread with SIGPIPE blocked.** The shell command and the environment commands write the topic (or the context's lines) to the child from a thread of their own, so the parent drains the child's output meanwhile (`env::feed_stdin`). A child that stops reading early — `head`, a program that fails — closes the pipe under the writer; with the signal blocked on that thread the write fails with EPIPE and the call takes its proper outcome (null, or a refusal over the context), where the default disposition `qua` restores for its own stdout would kill the whole process silently (exit 141). Found by feeding `@| u:head(-n=3)` a novel.

- **What a run is given is the run's, never the adapter's.** The invocation instant `now()` denotes, the shell gate, the quantifier and frontier bound overrides, reproducible provenance, the reference targets, the output collation and the sentence bonds travel in a `quarb::Run`; a tool parses a `Query` once (data-aware macros expand under the same `Run`) and runs it as often as it likes, each run answering an `Outcome` (the result, the closing table taken off for rendering, the unresolved references). The executor reads the options from the run in progress, and a run begun from inside another – an adapter answering through the engine, a model's constructor queries – inherits them, so nothing rides on an adapter method or a global. The adapter keeps only `quantifier_bound`, the depth its own structures need (metatheca's

chain), which a run may override. The convenience entries (`quarb::run` and its `_with_defs` / `_traced` twins) run under the ambient options: the instant the tool pinned, the bonds it set, the gate closed.

- **The root may be unnamed.** `name` returns `Option` precisely so an adapter can leave the root without a name, as the filesystem root `/` does.
- **A failed fetch is a refusal, never an empty answer.** The trait has no error channel below the entry points, so a read that fails inside an evaluation — a table load, a label listing, a relationship fetch, a count, a paginated listing cut off mid-way — records a refusal (`quarb::record_refusal`; `quarb::fetched` / `fetched_opt` wrap a `Result` that way) and the query errors with the adapter's message. A listing that failed is left uncached, so the next query reads again. The one legitimate emptiness is the server's own word — LDAP's `noSuchObject` and `insufficientAccessRights`.
- **Bounds where data set the size.** Every bound refuses with a message naming it: nesting (`MAX_NEST`, carried into holes, expansions and generated text), `repeat` / `indent` (`MAX_REPEAT_BYTES`, 64 MiB), outline levels (`MAX_OUTLINE_DEPTH`, 64 below the shallowest), unit exponents (`MAX_EXPONENT`, 32), mounts (`MAX_MOUNTS`, 254: `MountAdapter::try_new`), what an archive may decompress into memory (`MAX_ARCHIVE_BYTES`, 1 GiB, an xz stream checked against its index first). The executor's tree walks run on an explicit stack, so a source's depth is never the thread's.

3.2 The Source Boundary

Text is text inside the language; bytes meet it at two boundaries, both the engine's, in `quarb::source`. An adapter decides nothing about encodings.

- **Decoding** (`source::decode`, carried by a `SourceDecoding` per mount): `?encoding=LABEL` on the mount, then a byte-order mark, then the declaration the format's reporter finds in the bytes, then strict UTF-8 — a source that reaches the last step and is not valid UTF-8 is refused, naming the option. Nothing is guessed from bytes and nothing is decoded lossily; within a decided encoding a malformed sequence becomes U+FFFD and is counted. Labels are the Encoding Standard's (`encoding_rs`); the CJK tables ride the `cjk` feature of `quarb` (on for `qua`, off for the playground and the extension).
- **Reporters** are plain functions over bytes in the adapter crates: `quarb_xml::declared_encoding` (the prologue; the reader itself is `quarb::source::xml_declared_encoding`, shared with the packages), `quarb_html::declared_encoding` (the

meta charset prescan), `quarb_maildir::declared_encoding` (the MIME charset of a message's own headers). A host chooses the reporter by the source's name (`quarb_compose::declared_encoding`); the adapter never decodes.

- **Layers that read bytes** take the rule and answer for what they decoded: `Sourced` (the wrapper `qua` puts on a mount it read itself) answers `:::encoding` and `:::replaced` on the root; `ComposeAdapter` (`with_decoding`) reads a grafted leaf through its path and answers on that leaf; `ArchiveAdapter` (`open_with_decoding`) on each member it reads, tar and zip names included (an entry name is bytes under the rule, never the format's CP437 guess; zip members are read by index, so a decoded name need not match the library's); `MaildirAdapter` / `ImapAdapter` per message; `quarb_web::fs::open_dir_with_decoding` per page; `GitAdapter` / `MetathecaAdapter` (`open_with_decoding`) on a blob (one that decodes under none has no text; git's own structured output must be UTF-8, paths unquoted and messages transcoded by `--encoding=UTF-8`); the packages — docx, odt, epub — on each XML member through `parse_with_decoding` (the option, a byte-order mark, the member's XML prologue, else strict UTF-8; a member that decodes under none is an error naming it, never a part silently missing); `TreeSitterAdapter` / `CodeModel::open_with_decoding` and `quarb_text::read_conllu_dir_with` on a source file. A response body over HTTP is read whole under `source::text_from_reader` (a 256 MiB bound, strict UTF-8 — the wire formats are UTF-8 by their standards), never cut at a client library's cap nor replaced.
- **Delimited text** is decided the same way, never sniffed: `CsvAdapter::parse_declared` takes `?delimiter=` off the mount (`CsvOptions`), else an Excel `sep=X` first line, else the extension's default; a header without the decided delimiter but another candidate throughout is refused naming the option. Numbers in text read in the canonical form (`quarb::number`: digits, sign, `.`, exponent; grouped thousands under the default convention); a source that writes decimal commas declares `?decimal=comma` and its cells with a reading arrive in canonical form, so every reading downstream agrees. `| number(LOCALE)` reads text under a locale's convention from a small embedded table; `@| csv(";"; …)` takes the delimiter as a quoted first operand (the parser carries it as `delimiter=;`).
- **Dates in a locale's words**: `strptime(fmt; LOCALE)` / `tp` and `strftime(fmt; LOCALE)` / `tfmt` read and write the `%B %b %A %a` names from `quarb::datenames`, a small embedded CLDR table (by size of language: en, zh, es, ar, pt, ru, ja, de, fr, tr, it) listing every form a locale writes (the date form `октября`, the stand-alone `октябрь`, the abbreviations); every form parses, case folded, longest first, and writing takes the date form where the format carries a day (`%d`, `%e`) and the stand-alone form where it

does not (`%B %Y`: октябрь 1899). Format literals match as written (`'%d %B %Y г.'`). Without a locale the English names; a locale with no table refuses. The temporal reading of bare text stays ISO-8601.

- **Periods and time expressions** (spec: Time Expressions and the Timeline). `Value::Period` is a start second and a grain (`quarb::period`: year to second, ISO-8601 at reduced precision in print and in the interchange forms). The *calendar reading* (`Value::calendar_reading`) reads a period, reduced or full ISO text, an integer as a year, an instant at its printing grain. `value_eq` / `value_cmp` test for a period operand first and compare as intervals: nested periods are equal, disjoint ones ordered. A period has its own sort key (start, then the coarser first), no equality key (a search takes the scan), no numeric reading (arithmetic is null), and reads as its first instant wherever a command goes through the temporal reading. `period([GRAIN])` and `grain` are scalar commands. `timex(LOCALE)` (`quarb::timex`) is an expanding scalar command like `words`: one record per expression. Recognition is a list of templates per locale (`timexdata.rs`, the eleven locales of the date names), each a regular expression over slots (numeral, unit, month, weekday, day, year, clock, part of day) compiled once per locale. The earliest match wins, then the longest, then the rule listed first, and an overlapping match is dropped. Widening a list is the whole of teaching the reader an expression. No stemmer and no tagger takes part. Digits of other scripts, the typographic apostrophe and the Turkish dotted capital are folded one character for one before matching, so `at` counts characters of the text as written. A word boundary closes every template in the spaced locales, so an abbreviation's final point (`p.m.`, `г.`) stays with the sentence. `time_anchor(T)` and `@| timeline` share `period::anchor`: a span moves by calendar units keeping the anchor's grain (the month's last day where the target month lacks the day), except that an anchor finer than the day drops to the day when the offset names no unit finer than the day, a weekday offset and a time of day ask for a day or finer, a date with fields left out takes them from the anchor (under `forward`, the first such date at or after it). `timeline_command` is row-aligned like `shift`: it evaluates the expression and the anchor per capsa, folds the point over the context (a change of anchor value starts a chain), and adds `when` and `from` to each record. A value that is not a `timex` record is carried as `%(value; when; from)`.
- **Unit libraries**: `units LIBRARY;` in a model imports a kaiv `.faiv` library (path relative to the model file; `use` merges) into the unit table the unital reading consults for every source the model mounts: `ModelAdapter::unit_scale` answers from the imported definitions first (`kaiv::unit::scale_with`), then the source's own. Loaded at construction, refused by `check`: an unreadable or malformed file, the same file twice, a name the table holds already (built in, or an earlier

library). Unit names are letters in any script (the unital tokenizer admits Unicode letters), matched byte for byte as kaiv lists them; aliases read as their primary. A quantity prints its unit as written. Through kaiv-rs dev (0.15.0, path dependency) until kaiv publishes.

- **The output locale** (`quarb::locale`) governs rendering only, never the value: a tool decides the locale in force by its layers, nearest first — `--locale`, `QUARB_LOCALE`, `~/.quarb/config.toml` (`[output] locale`), the system's `LC_ALL` / `LANG` (`C` and `POSIX` canonical) — and renders what it prints through `OutputLocale`: numbers' decimal mark and grouping (CLDR's, with its minimum grouping digits), instants in the locale's medium date and time pattern through `strftime_in`, `@| csv`'s delimiter (the semicolon where the decimal mark is the comma), and `sort`'s collation when the query names none (`set_default_collation`, read by `default_text_order`; text alone is collated, numbers and instants keep their own order). Digits are grouped only when asked (`--grouping`, `QUARB_GROUPING=1`, `[output] grouping = true`): a year is not `1,876`. The engine itself never renders under a locale. A command is a value, the same wherever the query runs, so `@| csv` and `@| table` inside a pipeline are canonical; when one *closes* the query the tool renders it, as it renders `--csv` and `--table`: `Query::for_rendering` runs the query with the closing command replaced by an internal stand-in that yields the rows, and the `Outcome` carries the command's columns and named delimiter (`ClosingTable`). `@| json`, `@| jsonl` and `@| kaiv` stay canonical. A reader (`number`, `tp`) takes its locale from its operand alone, and refuses a locale it has no table for. `qua --diagnostics=info` reports the locale and its layer; the gates run with the environment's locale cleared. The session and wasm doors have no locale yet (the browser's `navigator.language` would be their system layer).

- **The order of text** (`stdlib::functions`: `sort_text_order`, `keyed_text_order`, `call_collator`) is one of three. With no locale in force, or where the call names `"C"` or `"POSIX"` (`code_point_order`), text orders by code point. Under a locale the call names (`sort`'s operand, `--locale=` on `sort_by`, `min_by`, `max_by`, `top`, `bottom`) or the output locale, it orders by that locale's collation in the locale's own case order: colligo's default since 0.2.0, lower case first but for the locales whose tailoring carries CLDR's `caseFirst upper`. The case options (`case_order`: `-u`, `-l`, `-i`) set the collator's case mode in place of the locale's, and with no collation in force stand on the root collation; beside `"C"` they are refused. The engine keeps no table of locales: the case order is colligo's to give. A keyed command reads each key once into the form it orders by (`rank_keyed`), as the bare `sort` does (`sort_keyed`); a comparison that read both keys afresh cost a sort of a million text keys seventeen seconds.

- **Streamed mounts** (`quarb-open`). A one-off query does not hold a CSV or TSV file (`CsvStream`: the rows by offset, one row parsed and kept) nor a `lines:` file of 256 KiB and more (`LinesStream`). A document of elements, the one array of a JSON document (`JsonArray`), a JSON lines file (`JsonLines`), the children of an XML document element (`XmlStream`), streams from `STREAM_DOCUMENTS_FROM` (128 MiB) or under `--stream`, and reads whole below it (`OpenOpts::stream_documents`): an element is parsed into its own arbor when a step reaches it and one is kept, so each step into an element parses it again, and an element alone costs two to three times its share of a whole parse; the stream wins where no element is read and loses where one is. Keeping every parsed element was tried and cost more memory than the whole read. All five read their file through `source::Window`, which holds the bytes around the place last read, growing while the reads sweep forward and falling back after a jump: a reader that seeks the file itself drops its buffer, and a fold over a million streamed rows was a seek and a read a row. A session, a run that keeps the arbor (`--kaiv`, `--save`, the resident server) and `--no-stream` mount whole.

- **The manual** (`syntax::man`) is two tables of the language repository, vendored by `scripts/vendor-commands.sh`: the command inventory (`commands.tsv`), from which the parser also reads the options a command takes, and the syntax index (`syntax-index.tsv`: a glyph, its name, its kind and what it does, in both languages). `man::man` answers a command's name, a glyph (as the index writes it, by its own marks, or written out: `Glyph::is_written_as`), a pipe (its page, then its commands) and `syntax [KIND]`; `man::line` reads a session's line that opens with `man`, which every host of a session takes (quai's REPL and `--run`, `QuaiSession::run` and `run_cell`, the kernel's cell), the browser's hosts having no shell and no colon commands. A manual line is no query: it takes no label.

- **Encoding** (`source::encode`): `@| encoding(LABEL)` is the output boundary and stands last. The engine has no byte value, so the tool takes the command off the end of the query, runs the rest and encodes what it would have printed (qua's `OutSink`); met anywhere else it is refused by the executor. The command before it decides what a character the encoding cannot hold becomes: after `@| json` / `@| jsonl` the format's `\u` escape (`source::Escape::Json`); after anything else the output is encoded while representable and refused at the first such character. Mail headers' RFC 2047 encoded words decode under their own charset through the same tables (`quarb-maildir`).

`bibref(STANDARD)` (`quarb::bibref`) reads one reference text under a citation standard — `gost`, `gb7714`, `cmos`, `ams`, an open tag set the command refuses outside of, naming it — into a record of the fields the grammar places without doubt (English names: `author`, `title`, `year`, `journal`,

pages, ...), its genus and the list label; a text that is no reference is null, as strptime answers for a text that is no date. Each grammar is a hand-written reader of the standard's punctuation (GOST's areas at . –, GB's type code after the title, Chicago's sentences and quoted titles, AMS's comma fields), the numbering prefix stripped first. The same module finds citations in prose (bracketed numbers and ranges with a pinpoint, author-year parentheses) and TextModel::lower_references (quarb-text) applies both for the ?references=STANDARD mount option: a paragraph or list item that parses becomes a bib node under the canonical campi (with the English aliases attached when no census is), its ::onym the written number, the list ordinal, or party and year; the citations become cit nodes and the resolution pass runs again.

3.3 Worked Example: the Filesystem Adapter

quarb-fs implements the live trait over a directory tree. It materializes the tree once, at construction, into arrays indexed by NodeId (path, children, parent); the trait methods are then trivial lookups:

```
impl AstAdapter for FsAdapter {
    fn root(&self) -> NodeId { self.root }
    fn children(&self, node: NodeId) -> Vec<NodeId> { self.children[
    node].clone() }
    fn name(&self, node: NodeId) -> Option<String> {
        if node == self.root { return None; }   // "/" is unnamed
        self.paths[node].file_name().map(|n| n.to_string_lossy().into()
    )
    }
    fn parent(&self, node: NodeId) -> Option<NodeId> { self.parents[
    node] }
}
```

Hidden entries and directory links [live]. An entry whose name starts with a dot carries the <hidden> trait (the mount root never does); such entries are in the tree only under --hidden. A symlink is a node with a target crosslink, and a symlink to a directory of the tree is also transparent to an explicit child step, as it is in the file system: with latest -> drafts, /latest/a.md is the node /drafts/a.md, the same one /latest->*/a.md reaches. The trait method behind this is child_host(node), the node whose children a child step reads; it defaults to the node itself, and the wrapping adapters (mounts, grafts, models, reflection) forward it. The descendant walk reads children alone, so // sees each file once and a cycle of links cannot trap it. A broken link, and a link whose target lies outside the mounted tree, stay leaves.

Ignore rules [live]. The materializing walk uses ignore::WalkBuilder — the same machinery ripgrep uses — so by default the adapter respects .gitignore/.ignore

files and skips hidden entries. This came directly from running the skeleton: `qua '//*.rs'` in the engine's own tree first returned a build artifact under `target/`, because nothing yet excluded it. The adapter now honors ignore rules by default, and `qua` exposes `--hidden` and `--no-ignore` to opt back in. The walk is currently eager (a targeted query still walks the whole tree); a lazy traversal with cascading ignore rules is a planned optimization. This is the pattern the guide follows: a concrete problem, surfaced by running the code, drives the next slice.

3.4 The Planned Surface

The rest of the arbor model, specified in the language but not yet built. Each becomes a live trait method (or a related capability) as its engine slice lands. The intended shapes:

Capability	Query	Adapter surface (sketch)
Pattern search	`=>`	`search(pattern) -> Vec<NodeId>`.

These signatures are provisional. They are recorded here so the direction is legible, not to freeze a contract — the filesystem adapter (and the JSON adapter after it) will reshape them as they are implemented.

Chapter 4

Building and Running

```
# from the engine workspace root
cargo build            # build the whole workspace
cargo test             # unit + integration + doc tests

# query a directory tree
cargo run -p qua -- '//*.rs' src/     # all Rust files under src/
cargo run -p qua -- '/README.md'      # a single top-level file

# query a document
cargo run -p qua -- '/users/0/name::' data.json
cargo run -p qua -- '//book[::pages > 200]:::id' catalog.xml
cargo run -p qua -- '//a::href' page.html
```

`qua <QUERY> [PATH]` runs `QUERY` against the directory `PATH` (default: the current directory) and prints each matching path on its own line, so it composes in shell pipelines alongside `sort`, `uniq`, and friends.

Chapter 5

Planned Integration Paths

Beyond the direct `AstAdapter` implementation shown above, these integration paths are described in the specification but **[planned]** (none is implemented yet):

- **Petgraph adapter.** A ready-made adapter over a `petgraph::Graph`, for Rust callers who transform their AST into a graph once and query it at speed. See `petgraph-adapter.md` in this project for the design note.
- **Python bindings.** A `quarb` Python package exposing the engine and an `AstAdapter` base class.
- **C FFI.** A C ABI for embedding the engine in other languages.

Each will be added once the core language surface is substantial enough to be worth binding.